

Highlights

What Works, What Doesn't, and Why: An Empirical Investigation of Cross-Project Bug Localization

A Eashaan Rao, Sridhar Chimalakonda

- Evaluates 3 CPL bug localization models under one identical protocol, 63 pairs.
- CPL beats within-project limited training for BLAZE and TRANSP-CNN, not COOBA.
- Cross-model direction agreement is only 44–62% on the same project pairs.
- Negative transfer risk ranges from 1.6% (BLAZE) to 31.7% (COOBA).
- CPL findings should be reported and validated per-architecture, not assumed general.

ARTICLE INFO

Keywords:

Bug Localization
Cross-Project Learning
Transfer Learning
Empirical Study
Software Maintenance

ABSTRACT

Bug localization, identifying the source location responsible for a reported defect, is a critical precursor to efficient debugging, yet the models that automate it require substantial within-project training data that newly active or small open-source projects rarely have. Cross-project learning (CPL) promises to close this gap by reusing labeled bugs from mature projects, but prior CPL studies evaluate a single model against ample-data within-project baselines that data-scarce projects cannot reach. We ask whether CPL's effectiveness is a general property of the transfer setting or contingent on model architecture, evaluating three architecturally distinct bug localization models, embedding-based dual-encoding (BLAZE), convolutional reranking (TRANP-CNN), and GNN-based adversarial adaptation (COOBA), across 63 source→target pairs from 13 open-source Python projects under identical bug corpora, data splits, retrieval candidates, and baselines, against within-project limited training (WP-small, ~10% of target labels), the data a newly active project actually has. CPL outperforms WP-small for 2 of 3 architectures: BLAZE with a large effect (Cohen's $d = 1.16$, win rate 95.2%, $p < 0.001$) and TRANP-CNN with a medium effect ($d = 0.65$, win rate 68.3%), while COOBA, in this corpus and under our re-implementation, shows only marginal gains ($d = 0.27$) with a 31.7% negative transfer rate. Cross-model direction agreement ranges from 44% to 62%: the same source→target pair can help one architecture and harm another, even though BLAZE and TRANP-CNN's gains are themselves correlated across pairs ($\rho = 0.516$, $p < 0.001$). CPL outcomes therefore vary substantially across architectures on the same source→target pairs; practitioners may need to validate CPL empirically for their chosen model rather than relying on results reported for a different one, and future CPL research should report results per-architecture rather than assume generalization.

1. Introduction

Bug localization, automatically identifying the source location(s) responsible for a reported defect, is a critical, labor-intensive activity in software maintenance. Deep learning-based models have demonstrated strong performance on established benchmarks (Lam et al., 2017; Liang et al., 2022; Ciborowska and Damevski, 2022; Hossain et al., 2024; Zhu et al., 2025), but their effectiveness depends on a readily-overlooked prerequisite: substantial labeled training data (paired bug reports and ground-truth source files) that a project accumulates only through years of sustained, tracked issue resolution.

For many projects this accumulation never reaches sufficiency or starts from zero. Early-stage or infrequently-reported projects may have only dozens to a few hundred verified mappings exactly when a localization tool is most needed. The difficulty is not merely data volume: developer familiarity with a codebase materially affects debugging efficiency (Wang et al., 2020), so a transferred model must compensate for exactly the contextual knowledge that familiarity provides. The scale is illustrated by our smallest-bug-corpus project, `pydata/xarray`, at only 110 verified mappings (distinct from our smallest-codebase project, `docker/compose` at 35K LoC with a larger 572-mapping corpus, so corpus size and codebase size are separate axes of “smallest” here). A standard limited-data split yields just 11 labeled training

examples, too few for reliable generalization and representative of any project in its first years of bug tracking, precisely where effective localization would be most valuable.

Cross-project learning (CPL) is a natural response: train on a data-rich source project, then apply or fine-tune on the data-scarce target, on the intuition that the bug-report-to-code relationship may transfer across projects sharing a language or coding style. Several models demonstrate this, including TRANP-CNN (Huo et al., 2021), COOBA (Zhu et al., 2020), BL-GAN (Zhu et al., 2023), and BLAZE (Chakraborty et al., 2025). Yet a fundamental question remains unanswered: *does CPL effectively help the practitioners it is designed to serve?* Each of these studies shows its model outperforming an ablated baseline or earlier CPL method, but their within-project comparators use the *majority* of the target's labeled history; data-scarce practitioners cannot match this volume, so these experiments establish that CPL surpasses a standard they cannot reach, never whether it beats the limited training they *can* perform. In the analogous task of cross-project defect prediction, a 90-study survey found transfer context-dependent, helping in some conditions and hurting in others with no single reliable predictor (Hosseini et al., 2019; Zhang et al., 2022). Whether CPL helps teams with limited within-project data is, to our knowledge, unanswered in the bug localization literature.

A second gap compounds this: to our knowledge, every published CPL paper evaluates a single model of its own design, so it is unknown whether demonstrated benefit is a property of the cross-project paradigm or an artifact of one model's inductive biases. A practitioner reading one CPL paper and choosing a different tool cannot know whether its findings apply. No existing study, to our knowledge, uses

* This work was supported by Indian Institute of Technology Tirupati.

*Corresponding author

✉ cs21d002@iittp.ac.in (A.E. Rao); ch@iittp.ac.in (S. Chimalakonda)

ORCID(s): <https://orcid.org/0000-0001-8499-7538> (A.E. Rao);
<https://orcid.org/0000-0003-0818-8178> (S. Chimalakonda)

multiple CPL models as independent instruments across the same project pairs.

We fill both gaps with a controlled empirical study across three architecturally distinct, well-established CPL models spanning the dominant paradigms (embedding-based BLAZE, GNN-based COOBA, CNN-based TRANSP-CNN) on 63 directional source→target pairs from 13 open-source Python projects in four domains (Data Science/ML, Developer Tools, Systems/Cloud, Applications), under four scenarios isolating the contribution of cross-project knowledge. The primary baseline throughout is *within-project limited training*, the data a newly active project actually has, not prior work's ample-data ceiling. A core methodological contribution, independent of empirical findings, is the evaluation protocol itself: since each prior paper proposes its own architecture, corpus, and comparator, cross-paper comparisons conflate architectural differences with varying conditions, making it difficult to attribute outcomes to any specific cause. Here, all three models share the same 63 pairs, bug reports, train/test splits, FAISS indexes, and primary baseline, treated as **independent scientific instruments under an identical experimental protocol**, supporting attribution of outcome differences to architecture rather than incidental conditions, though not by itself establishing the causal mechanism behind any given outcome.

Answering this rigorously presents three challenges. **(Ch1)** Attributing outcomes to architecture requires eliminating heterogeneity in training data, evaluation corpora, and baselines; to our knowledge, no prior CPL study has enforced this. **(Ch2)** Choosing the baseline is conceptually subtle: data-scarce practitioners cannot reach prior work's ample-data ceiling, so any comparison against it answers the wrong question. **(Ch3)** Assembling a corpus large enough for statistical power across diverse pairs requires a principled pair-selection strategy, since a naïve all-pairs design would exceed any realistic compute budget.

Contributions and Summary of Findings

What we contribute. (C1) A controlled multi-instrument evaluation framework: three architecturally distinct CPL models evaluated under an identical protocol; same 63 source→target pairs, bug reports, data splits, FAISS indexes, and primary baseline. To our knowledge, no prior CPL study provides this. **(C2)** To our knowledge, the first systematic comparison of CPL against within-project *limited* training: the data a newly active project actually has, not the ample-data ceiling used in prior work. **(C3)** A negative-transfer risk spectrum across architectures under identical conditions, 1.6% of pairs for BLAZE, 15.9% for TRANSP-CNN, 31.7% for COOBA, not previously reported in the CPL literature and directly actionable for deployment risk assessment.

Summary of findings. CPL benefit varies substantially across the three models tested on identical data: large for BLAZE, medium for TRANSP-CNN, and marginal for COOBA; Section 4 reports the full per-research-question breakdown (win rates, effect sizes, negative-transfer risk, cross-model agreement). The divergence is consistent with a single mechanistic axis, whether a model's ranking mechanism requires target-side calibration at inference time, which Section 5.2 develops in detail.

The rest of the paper is organized as follows. Section 2 reviews related work on bug localization and CPL. Section 3 describes the study design. Section 4 presents results for each research question. Section 5 discusses practical implications and threats to validity. Section 6 concludes.

2. Background and Related Work

2.1. Bug Localization

Bug localization ranks source files (or functions) by their likelihood of containing the defect described in a natural-language bug report. Early approaches relied on information-retrieval similarity between bug report text and source file content (Zhou et al., 2012; Saha et al., 2013; Ye et al., 2014), using measures such as BM25 (Robertson and Zaragoza, 2009) or VSM over code tokens, simple and training-free, but limited by vocabulary mismatch between bug report prose and code identifiers.

Deep learning improved on IR baselines by learning joint representations of bug reports and code: Lam et al. (2017) (DNNLOC) combined textual similarity, file change history, and structural features in a multi-layer network, improving over IR on Java benchmarks; Liang et al. (2022) extended this to function-level localization via a call-dependency graph neural network; Ciborowska and Damevski (2022) replaced conventional encoders with pre-trained BERT (Devlin et al., 2019) representations over code changesets, showing that transfer from large pre-trained models reduces the labeled-data requirement. A recent survey (Niu et al., 2025)

confirms labeled within-project data remains the standard assumption across this literature. The Lee et al. (2018) benchmark (51 Java projects) is a reference point for comparing methods (Bogomolov et al., 2024), though a systematic review of debugging benchmarks cautions that benchmark choice materially affects reported performance and cross-study comparability (Hirsch and Hofer, 2022). Careful pre/post-processing (stack-trace filtering, stop-word removal, code-structure weighting) can itself yield substantial gains independent of the underlying ranking model (Kim et al., 2022).

A consistent limitation of all within-project approaches is assuming the target has accumulated sufficient labeled bug reports for training, an assumption that fails for new or data-scarce projects, the exact regime motivating this study.

2.2. Cross-Project Bug Localization

Cross-project bug localization (CPL) addresses data scarcity by transferring knowledge from a source project to a target. Huo et al. (2021) proposed TRANP-CNN, the first deep learning CPL model, a convolutional network learning shared bug-report-code representations from a source project and applying them to a target, but evaluated only against an ample-data within-project baseline, leaving the practically relevant data-scarce comparison unanswered (Zhu et al., 2021). Zhu et al. (2020) introduced COOBA, using adversarial domain adaptation (Ganin et al., 2016) via a gradient reversal layer to learn domain-invariant AST-structured code representations; in our re-implementation this adversarial objective required target-side signal at training time and collapsed to near-zero performance without it. Zhu et al. (2023) proposed BL-GAN, a semi-supervised approach augmenting scarce target labels with generative adversarial networks, but does not test whether CPL helps when the practitioner has only limited within-project data as the alternative. Chakraborty et al. (2025) proposed BLAZE, a dual-encoder contrastive model using dynamic chunking and hard-negative mining, evaluating cross-project and cross-language transfer but testing only a single architecture, leaving cross-model CPL consistency unknown (Chandramohan et al., 2024; Li et al., 2025; Reddy et al., 2025).

A critical gap remains: each paper proposes and evaluates its own model, typically against within-project ample-data baselines or earlier CPL methods. To our knowledge, *no existing study* (i) uses multiple CPL models as independent instruments, (ii) evaluates against the practitioner-relevant limited within-project baseline, (iii) characterizes the conditions governing CPL success at scale, or (iv) tests cross-model consistency of findings.

A distinct emerging paradigm employs large language models in agentic workflows, such as SWE-agent (Yang et al., 2024b) and Agentless (Xia et al., 2024; Zhang et al., 2024; Hossain et al., 2024), bypassing labeled-data requirements by leveraging pre-training directly for fault localization and patch generation. The relationship to the CPL setting studied here rests on a task difference, not a deployment-scenario one: agents select and patch a small shortlist of files

for a specific issue, whereas this study measures fine-grained ranking across an entire codebase (Top-1, Top-5, Top-10, MRR, MAP). On SWE-bench, agent file-localization accuracy is an intermediate step toward patching rather than a reported output, so no direct comparison is available from existing agent evaluations, and we did not run one ourselves. Our corpus is also drawn largely from SWE-bench (11 of 13 projects, Section 3.3), the same popular, well-instrumented, test-suite-bearing repositories agents are typically evaluated on, so this study does not itself demonstrate a setting complementary to agents; privacy-constrained deployment, offline inference, and cost-sensitivity are motivations for studying supervised CPL rather than findings established here. The conditions governing CPL success identified in this study, domain alignment, embedding quality, and inference-time adaptation, may nonetheless inform how agents prioritize code during fault localization.

These claims are made relative to a targeted search rather than a systematic review. We seeded from four supervised deep-learning CPL bug-localization papers (TRANP-CNN, COOBA, BL-GAN, BLAZE), selected for codebase availability and evidence of being tested for cross-project transfer. TRANP-CNN's and COOBA's original implementations were not available and were built from scratch for this study (Section 3.2). We performed backward and forward snowballing on each seed paper, ran keyword queries combining cross-project and cross-language with bug localization and fault localization on the ACM Digital Library and IEEE Xplore, and cross-checked coverage against a recent survey of deep-learning IR-based bug localization (Niu et al., 2025). We screened for work that trains a localization model on one project's labeled bugs and evaluates it on another. Within that set, none of the four properties above is satisfied by a prior study. We may have missed work outside these venues or using different terminology.

2.3. Transfer Learning in Software Engineering

The broader literature on cross-project transfer in software engineering provides useful context (Pan and Yang, 2010). Cross-project defect prediction (predicting module defectiveness using another project's training data) has been studied systematically for years: Hosseini et al. (2019) survey 90 such studies and find transfer context-dependent, helping in some conditions and hurting in others with no single reliable predictor (Sotto-Mayor and Kalech, 2024). Proposed remedies span feature-level transfer, e.g. matching source/target feature distributions before training a cross-company classifier (Yu et al., 2017), and adaptive instance weighting accounting for feature and class distribution shift jointly (Tong et al., 2023), though a systematic review finds model architecture interacts with these strategies in ways not yet well characterized (Giray et al., 2023), a gap this study addresses directly for bug localization, motivating our characterization of CPL success conditions and focus on negative transfer as a deployment risk. The parallel is informative: like defect prediction, bug localization is a

within-project learning task where CPL is possible but not guaranteed beneficial.

The theoretical basis for COOBA's adversarial domain adaptation is domain-adversarial neural networks (Ganin et al., 2016), which minimize feature-level source-target divergence by training a domain discriminator with reversed gradients during feature extraction, widely applied in software engineering transfer settings, though its effectiveness for cross-project code understanding depends on whether the feature extractor captures genuinely domain-invariant representations, an assumption we test empirically.

2.4. Pre-trained Code Embeddings: Design Motivation

Recent advances in pre-trained code embeddings have raised representation quality for both within- and cross-project models (Feng et al., 2020; Guo et al., 2021; Wang et al., 2021; Guo et al., 2022). This study uses BAAI/bge-code-v1 (Beijing Academy of Artificial Intelligence, 2025) (Section 3.4) as a shared first-stage retrieval backbone across all three models, so the reranked candidate pool is identical and reranking differences are attributable to the models themselves. BLAZE additionally uses BGE as its reranking backbone, while TRANP-CNN and COOBA rerank with their original designs' input representations (learned token embeddings and GloVe-300 with AST graphs, respectively; Section 3.2).

Together, these four subsections frame the design choices in Section 3: the shared BGE retrieval stage controls the candidate pool, the three architecturally distinct models serve as independent instruments for testing CPL consistency, and the limited within-project baseline addresses the evaluation gap identified above.

3. Study Design

Figure 1 summarizes the end-to-end design. The study is organized around a single methodological principle: making CPL benefit attributable to the cross-project paradigm and to model architecture, rather than to incidental differences in training data, candidate pools, or evaluation conditions. Every design choice below, the shared retrieval backbone, identical datasets and held-out splits, the four-scenario training framework, the primary baseline, and the structured pair selection, follows from and is justified by this principle.

A **FAISS IndexFlatIP** (Johnson et al., 2021) retrieval stage, built from identical BGE embeddings across all target projects, serves as the first-stage candidate pool shared across all three models. As argued in Section 1, prior CPL papers each propose one model with its own corpus and comparator, conflating architecture with experimental conditions; we depart from this by treating three architecturally distinct models as **independent scientific instruments**, all evaluated under an identical protocol: the same projects, bug reports, train/test splits, FAISS retrieval indexes, evaluation metrics, and primary baseline. This supports attributing outcome differences to architecture, not just documenting

that outcomes differ, though it does not by itself establish mechanism.

Reporting standards. This study follows the ACM SIGSOFT Empirical Standards (Ralph et al., 2021), specifically the General Standard and the Data Science Standard. A completed checklist against both standards is included in the replication package (see Data Availability).

3.1. Research Questions

This study is organized around three research questions, each targeting a distinct aspect of CPL viability.

RQ1 *Does cross-project learning outperform within-project training under label scarcity?* Tests the core CPL premise against the practitioner-relevant baseline (Section 1): the limited within-project data a project actually has, not prior work's ample-data ceiling.

RQ2 *What project-level characteristics determine whether CPL succeeds or fails?* Even if beneficial on average, practitioners need to know *when* CPL is and is not appropriate: moderating conditions (target size, domain alignment) and the risk of negative transfer.

RQ3 *How consistent are CPL outcomes across different model implementations?* A paradigm-level benefit should be consistent across architectures; a model-specific one would not generalize. Also asks whether project metadata reliably guides source selection.

Hypotheses and motivated expectations. RQ1 is confirmatory, a directional claim against a quantifiable baseline under a controlled protocol, so we state formal hypotheses:

H_0 : CP-transfer does not outperform within-project limited training: $MRR_{CPT} \leq MRR_{WPS}$.

H_1 : CP-transfer outperforms within-project limited training: $MRR_{CPT} > MRR_{WPS}$ (one-sided).

This direction is motivated by CPL literature demonstrating transfer feasibility (Huo et al., 2021; Chakraborty et al., 2025; Zhu et al., 2020), though all prior comparisons use ample-data baselines; whether H_1 holds against the baseline data-scarce practitioners can actually reach is precisely what has not been tested.

RQ2 and RQ3 are exploratory, so we state literature-grounded expectations rather than formal hypotheses. For **RQ2**, cross-project defect prediction research finds transfer success context-dependent with no single reliable predictor (Hosseini et al., 2019); we expect analogous conditions for CPL but make no a priori prediction about which moderating factors dominate or whether negative-transfer rates are uniform across architectures. For **RQ3**, since we are not aware of a CPL study evaluating more than one model, cross-architecture consistency is empirically unknown; we expect outcomes to be at least partially architecture-mediated, given the fundamental inference-time differences across BLAZE, COOBA, and TRANP-CNN, but make no prediction about the degree of divergence or whether source quality metrics will transfer across architectures.

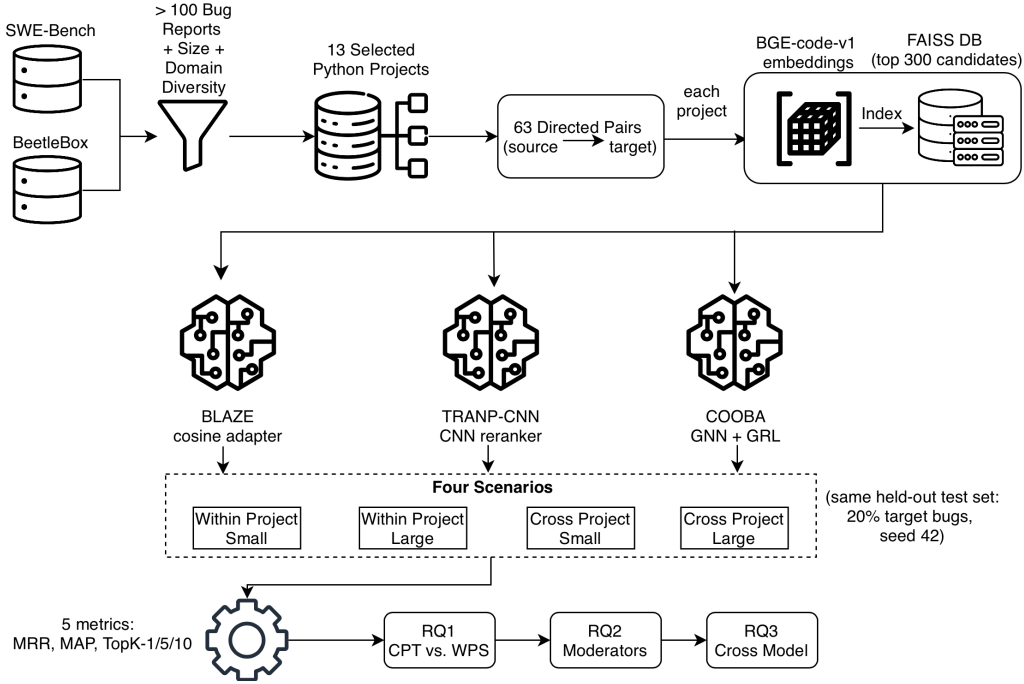


Figure 1: End-to-end study design. Python projects from SWE-bench and BeetleBox are filtered by bug-report volume and diversity criteria to yield 13 projects, which are paired into 63 directed pairs via structured selection. All target projects share one BGE-code-v1 embedding and one FAISS top-300 retrieval stage (the controlled backbone). Three architecturally distinct models (BLAZE, TRANSP-CNN, COOBA) rerank the same candidate pool in four training scenarios, with the held-out test set held constant throughout. The uniform protocol makes CPL outcomes attributable to model architecture rather than experimental heterogeneity.

3.2. Benchmarked Models

A central design choice is evaluating CPL with three architecturally distinct models rather than a single representative, guarding against conclusions that are artifacts of one model’s inductive bias. Each dominant paradigm is represented by exactly one model: BLAZE (embedding-based), COOBA (GNN-based), TRANSP-CNN (CNN-based). Where the paper refers to “embedding-based,” “GNN-based,” or “CNN-based” behavior, this is shorthand for the specific model tested, not a claim about the paradigm generally; whether another model of the same paradigm behaves the same way is untested here.

BLAZE (Chakraborty et al., 2025). A dual-encoder contrastive model (Reimers and Gurevych, 2019) that retrieves the top-300 candidates via the shared FAISS index and re-ranks by maximum cosine similarity. Its backbone is BAAI/bge-code-v1 (Beijing Academy of Artificial Intelligence, 2025), frozen during training, with two modal-specific residual adapter modules trained via in-batch NTX-entLoss (van den Oord et al., 2018) and hard-negative/hard-positive scaling, one for bug reports (prefixed `search_query:`), one for code chunks (prefixed `search_document:`). Each file is scored at evaluation time as the maximum cosine similarity over its 512-token chunks (MaxSim), making the architecture sensitive to the most relevant passage within a long file, advantageous for large codebases.

COOBA (Zhu et al., 2020). A graph neural network encoding source files as AST graphs with GloVe-300 node features via graph convolutional layers (GCNConv) and global mean pooling; bug reports are encoded by a bidirectional LSTM over GloVe-300 tokens, faithful to the original design. A shared feature extractor over these text and AST representations is optimized to minimize ranking loss while a Gradient Reversal Layer (Ganin et al., 2016) before a binary project discriminator prevents it from distinguishing source from target representations. This adversarial objective is the core CPL mechanism. COOBA reranks the shared FAISS top-300 pool.

TRANP-CNN (Huo et al., 2021). A CNN-based reranker processing (bug report, candidate file) pairs through parallel convolutional feature extractors (kernel sizes 3, 4, 5) into scalar relevance scores, trained with margin ranking loss on FAISS top-300 candidates. Inputs are tokenized with the BGE tokenizer vocabulary but mapped through embedding layers trained from scratch, following the original design’s learned-representation approach. The earliest deep learning CPL approach in our benchmark, it provides a third architectural perspective, convolutional text-code alignment, complementing BLAZE’s embedding-based and COOBA’s graph-structural paradigms.

Table 1 summarizes the three models’ key properties, together spanning the dominant implementation paradigms in current CPL literature and providing a basis for testing

Table 1

Summary of the three benchmarked bug localization models.

Model	Architecture	CPL Mechanism	Retrieval
BLAZE (Chakraborty et al., 2025)	Dual-encoder (BGE adapters)	Contrastive adapter fine-tuning	FAISS top-300
COOBA (Zhu et al., 2020)	GNN on AST graphs	Adversarial domain adaptation	FAISS top-300
TRANP-CNN (Huo et al., 2021)	CNN reranker	Shared representation learning	FAISS top-300

Table 2Training configuration used for all 756 experiment runs. LR = learning rate; effective batch = batch size $\times$ gradient-accumulation steps. Early stopping monitors validation loss.

Setting	BLAZE	COOBA	TRANP-CNN
Optimizer	AdamW	AdamW	AdamW
Learning rate	1×10^{-4}	1×10^{-3} (discriminator 5×10^{-4})	1×10^{-3}
Weight decay	10^{-2}	10^{-5}	10^{-5}
Effective batch	8 (4×2)	16	192 (96×2)
Max epochs	10	10	20
Stopping	Early stop, patience 3	Fixed epoch budget	Early stop, patience 3
Loss	NTXent ($\tau = 0.3$)	Margin ranking + adversarial ($\lambda_{adv} = 0.1$)	Margin ranking
LR schedule	Linear, 10% warm-up	Constant	Constant

whether CPL viability is a property of the paradigm or of specific architectural choices.

Adaptations and training configuration. TRANP-CNN and COOBA predate the retrieval-augmented setup used here and were re-implemented for this study, architecturally faithful to their original publications (loss functions, encoders, adaptation mechanisms preserved) with two deliberate departures: all three models rerank the shared FAISS top-300 pool, and TRANP-CNN tokenizes with the BGE vocabulary while training its embedding layers from scratch. Absolute scores are therefore not directly comparable to the original papers' (Section 5.4 discusses the construct implications). Table 2 lists each model's training configuration, held fixed across all scenarios and pairs.

Throughout, "architecture" means the bundle distinguishing the three models: input representation (BGE for BLAZE, GloVe-300 text/AST for COOBA, learned tokens for TRANP-CNN), ranking mechanism (embedding similarity, adversarially-adapted GNN score, CNN scalar score), and adaptation mechanism (frozen backbone with adapters, domain-adversarial training, standard fine-tuning). These dimensions co-vary with publication year (BLAZE 2025, COOBA 2020, TRANP-CNN 2019) and provenance (BLAZE as published, the other two re-implemented); Section 5 treats "architecture" as a bundle and attributes outcome differences to it only as an interpretation, not a controlled causal claim, since these dimensions are not independently manipulated here.

3.3. Dataset Selection and Curation

Source Benchmarks. Projects were drawn from two established Python benchmarks: **SWE-bench** (Jimenez et al., 2024), contributing 11 of the 13 final projects with curated, verified bug-report-to-commit-SHA ground truth across a

broad domain range, and **BeetleBox** (Chakraborty et al., 2025), contributing the remaining 2 (ansible, localstack) to extend coverage to large-scale infrastructure repositories absent from SWE-bench. Together they provide the depth of verified ground truth needed for controlled CPL evaluation (Kalliamvakou et al., 2014).

Selection Criteria. A project was retained if it had (i) *bug report volume*: at least 100 unique bug reports with verified ground-truth file mappings, after discarding reports with missing annotations, ambiguous multi-file patches (> 10 files), or non-source patches. Two further conditions are properties of the resulting *set*, verified post hoc rather than filtered per project: (ii) *codebase size diversity* spanning at least one order of magnitude, so size is a meaningful RQ2 moderator; and (iii) *domain coverage* including at least one project per target domain class, so no single application type dominates. Both held for the set produced by criterion (i) alone, so no project was added or removed to satisfy them. Phase 1 restricts to Python to establish a language-homogeneous baseline before cross-language extension (Section 6).

Phase 1 Scope: 13 Python Projects. Applying criterion (i) yields **13 qualifying projects**, the **complete** qualifying set, none excluded after filtering. The corpus spans meaningful variation: codebase size 35K–572K LoC (one order of magnitude), four application domains, and bug corpus size 110–1,336 reports (Table 3).

The Python-only scope is also motivated by compute: 63 pairs $\times$ 4 scenarios $\times$ 3 models is **756 independent experiment runs**, each involving training from scratch, FAISS index construction, and per-bug evaluation, approximately five to six months of wall-clock time on NVIDIA L40S

Table 3

The 13 Python projects used in Phase 1. Size categories: Small (<100K LoC), Medium (100–300K LoC), Large (>300K LoC). Source indicates the benchmark from which each project's bug-report-commit ground truth was drawn. Recall@300 is the fraction of bug reports for which the ground-truth file is retrieved within the shared FAISS top-300 candidate pool when this project is the target; it bounds every absolute metric for all three models, which rerank that same pool (Section 5.4).

Project	Domain	LoC	Bug Reports	Size	Source	Recall@300
docker/compose	Systems / Cloud	35,000	572	Small	SWE-bench	99.8%
jupyterlab/jupyterlab	Data Science / ML	39,000	197	Small	SWE-bench	47.0%
lightning-ai/lightning	Data Science / ML	51,000	366	Small	SWE-bench	98.9%
ipython/ipython	Developer Tools	78,000	832	Small	SWE-bench	74.6%
prefecthq/prefect	Data Science / ML	107,000	404	Medium	SWE-bench	69.7%
mesonbuild/meson	Developer Tools	121,000	937	Medium	SWE-bench	45.5%
pydata/xarray	Data Science / ML	142,000	110	Medium	SWE-bench	100.0%
numpy/numpy	Data Science / ML	277,000	789	Medium	SWE-bench	74.5%
wagtail/wagtail	Applications	286,000	404	Medium	SWE-bench	34.6%
ansible/ansible	Systems / Cloud	348,000	768	Large	BeetleBox	31.3%
scikit-learn/scikit-learn	Data Science / ML	376,000	228	Large	SWE-bench	62.3%
qiskit/qiskit	Applications	435,000	1336	Large	SWE-bench	56.1%
localstack/localstack	Systems / Cloud	572,000	472	Large	BeetleBox	76.5%

GPUs. Extending to Java would more than triple this budget, so it is scoped as a future phase.

Domain Classification Protocol. Each project's domain (Table 3) was assigned by the authors from its README and predominant use case, into one of four categories. Some calls are genuinely contestable: jupyterlab/jupyterlab is Data Science/ML on the strength of its use case (an interactive data-science computing environment) despite a frontend-heavy codebase (Section 5 discusses this tension for COOBA specifically); ipython/ipython is Developer Tools, being a general-purpose shell rather than domain-specific; and prefecthq/prefect is Data Science/ML as a workflow tool whose user base centers on ML/data pipelines. This classification lacked an independent second rater or agreement statistic, noted as a limitation in Section 5.4; a second pass with one (e.g. Cohen's κ) should precede treating the domain-dependent RQ2 findings (Table 9) as final.

Pair Selection: 63 Directional Pairs. A naïve all-pairs design over 13 projects yields 156 directed pairs (1,872 runs), infeasible within our compute budget without sacrificing statistical quality. We instead define a principled **structured 63-pair set** in three groups (Table 4): **Group A** (30 pairs), all bidirectional pairs among the six Data Science/ML projects (jupyterlab, lightning-ai, prefect, xarray, numpy, scikit-learn), the statistical backbone for controlled within-domain comparison and commutativity analysis; **Group B** (14 pairs), both directions between jupyterlab (39K LoC, second-smallest after docker/compose at 35K) and each of the seven non-DS projects, isolating domain crossing from target-size confound in the seven jupyterlab-as-target directions while the reverse directions give complementary large-target observations; and **Group C** (19 pairs), strategically selected cross-domain pairs filling remaining gaps (large-to-large transfers, Systems/Applications interactions, and

ensuring every target has at least three candidate sources for RQ3's source-selection analysis).

3.4. Data Preprocessing

The preprocessing pipeline runs once per project and persists outputs as project-level databases, avoiding redundant recomputation across the 756 experiment runs.

Bug Report Curation. Raw bug reports are extracted from the two benchmarks and cross-referenced against commit history for ground-truth file mappings, uniquely identified by commit SHA; reports with missing SHAs, ambiguous patches (> 10 files), or non-source patches are discarded. The resulting bug metadata database contains {bug_id, bug_text, ground_truth_files, commit_sha}, where ground_truth_files is a list (Section 3.7 discusses the multi-file metric implications).

Source Code Embedding. Source files are extracted at each bug report's filing-time commit and encoded with **BAAI/bge-code-v1** (Beijing Academy of Artificial Intelligence, 2025), a 1536-dimensional Qwen2-based decoder pre-trained for code understanding; files exceeding 512 tokens are split into overlapping chunks (512 tokens, 50-token overlap) and their embeddings averaged into a single file-level float32 vector, with bug report texts encoded analogously. Producer-consumer multiprocessing (8 CPU workers) with batched GPU inference (batch size 256) processes all 13 projects in approximately 6 hours on a single NVIDIA L40S GPU.

Per-project outputs are two binary databases, a *blob embedding database* (blob SHA → embedding) and a *bug metadata database* (bug ID → embedding and ground truth), from which all three models draw identical first-stage retrieval candidates.

Table 4

Structure of the structured 63-pair set.

Group	Primary Purpose	Pairs
A, Within-Domain (DS×DS)	RQ1 statistical backbone; commutativity	30
B, Cross-Domain Hub (jupyterlab)	Domain-crossing; target-size isolation	14
C, Strategic Cross-Domain	Large-target coverage; source selection	19
Total	252 runs/model; 756 runs total (63 pairs)	63

3.5. Evaluation Pipeline

Retriever-Reranker Pipeline. All three models share a two-stage pipeline. Retrieval builds a **FAISS IndexFlatIP** index from L2-normalized BGE file embeddings (inner product = cosine similarity) and queries the top- $K = 300$ candidates per bug report; recall at $K = 300$ varies across projects (Section 5.4), and ground-truth files missed here are missed at reranking. In reranking, TRANP-CNN and COOBA score (bug report, candidate file) pairs directly, while BLAZE re-ranks the top-300 by maximum cosine similarity between the fine-tuned bug report embedding and each file's chunk embeddings (MaxSim). The raw FAISS ranking itself is weak (mean MRR = 0.029; Table 6), largely because averaging chunk embeddings dilutes long files' signal. Its role is candidate *recall*; ranking precision is the rerankers' task.

Why Dense Retrieval, Not Lexical IR. The shared candidate-generation stage uses dense BGE embeddings rather than a classical method like BM25 (Robertson and Zaragoza, 2009), primarily for representational consistency across the three rerankers rather than lexical matching's vocabulary-mismatch limitation alone (Section 2): BLAZE's own reranking already operates in this embedding space (MaxSim over BGE chunks), so a shared dense backbone avoids a second, uncontrolled representation mismatch on top of those discussed in Section 3.2. Dense embeddings also capture semantic/structural similarity beyond lexical overlap, relevant for bug reports describing symptoms without naming the implicated file, consistent with the broader shift toward pretrained code embeddings as the standard retrieval substrate. This choice concerns candidate generation, not the cold-start comparison: we did not evaluate classical IR as a zero-label comparator in its own right, so the cold-start viability result (Section 4) is relative to WP-small, not the full set of zero-label options, including lexical IR, open to an unlabelled project.

3.6. Training Scenarios

The four training scenarios are the study's core methodological contribution, addressing a specific evaluation gap: every prior CPL paper compares against a within-project baseline trained on the *majority* of target labels, a ceiling data-scarce practitioners cannot reach, so beating it tells them nothing useful. Instead, the four scenarios form a clean **ablation over the training-time supervision signal**, from none, to limited target data only, to limited target data plus

full source data, to the ample-data ceiling, with the test set held constant throughout (20% of target bugs, seed 42, identical across models) so only training composition changes, isolating the marginal value of each supervision source.

- **WP-small (WPS)** (Within-Project, Limited): trained on ~10% of the target corpus, drawn from the same 80% pool as WP-large (nested; neither touches the 20% test set), the data a newly active project actually has, ranging from 11 labeled bugs (pydata/xarray) to 133 (qiskit/qiskit), a small-data regime even at its upper end. This is the *primary baseline* throughout, a deliberate departure from prior work's ample-data comparator: measuring CPL's benefit against a data-scarce baseline tests whether it delivers on its design intent.
- **WP-large (WPL)** (Within-Project, Ample): trained on 80% of the target corpus, the ceiling achievable with maximum within-project supervision. Not a CPL comparator but a reference: WP-large – CP-transfer quantifies what CPL still cannot achieve, while CP-transfer – WP-small quantifies its gain over the practical baseline.
- **CP-cold-start (CPC)** (Cross-Project, Zero-Shot): trained on 100% of source bugs with *zero* target annotations, the minimum viable deployment for a project at day zero, testing whether a source-trained model can be applied with no annotation effort.
- **CP-transfer (CPT)** (Cross-Project, Fine-Tuned): jointly trained on 100% of source bugs plus ~20% of target bugs (source and target batches in the same training step, not a sequential pretrain-then-fine-tune schedule). This target-side share is twice WP-small's (~20% vs. ~10%); the two scenarios are *not* budget-matched, so a CP-transfer advantage cannot be attributed to cross-project knowledge alone. It may partly reflect the larger target-side set on top of the extra source data. Section 5.4 treats this as an internal-validity threat; the CP-transfer-vs-WP-small gains reported throughout should be read as an upper bound on the cross-project-knowledge component, not an isolated measurement of it.

This framework applies identically to all three models, with no model-specific adjustments, what makes the RQ3

Table 5

Four evaluation scenarios with data configurations and roles. All share the same held-out test set (20% of target bugs, seed 42).

Scenario	Training Data	Target Budget	Role in Study
WP-small	~10% target bugs only	Low	Primary baseline; practical data-scarce setting
WP-large	80% target bugs only	High	Performance ceiling
CP-cold-start	100% source, 0% target	None	Zero-shot deployment viability
CP-transfer	100% source + ~20% target	Medium	Core CPL scenario; target share double WP-small's (Section 5.4)

cross-model comparison valid and supports attributing effectiveness differences to the models themselves rather than heterogeneous conditions. Table 5 summarizes.

3.7. Evaluation Metrics

Five complementary metrics provide a complete picture of retrieval breadth and ranking precision, computed over the re-ranked candidate list for each test bug report and averaged over all test reports.¹

Top- K Recall ($K \in \{1, 5, 10\}$): fraction of bug reports for which the ground-truth file appears within the top K positions. Top-1 captures precision-at-one. Top-10 captures shortlist breadth.

A bug report's ground truth can span multiple files (patches touching up to 10 files are retained, Section 3.3), so MAP and MRR are computed under the standard multi-relevant-document definitions rather than reducing to a shared single-file formula.

Mean Reciprocal Rank (MRR): for each bug report, the reciprocal rank of the *best-ranked* ground-truth file, $1/\min_j r_{i,j}$ over its ground-truth files j , averaged across all bug reports i .

Mean Average Precision (MAP): for each bug report with ground-truth files at sorted ranks $r_{i,1} \leq r_{i,2} \leq \dots \leq r_{i,k}$, the average precision $\frac{1}{k} \sum_{m=1}^k m/r_{i,m}$, averaged across all bug reports. MAP and MRR coincide only for the subset of bug reports with a single ground-truth file; Table 6 reports both because most bug reports in this corpus have more than one.

MRR and MAP diverge from Top- K when the correct file is retrieved but ranked outside the top- K shortlist. This divergence is itself informative. It reveals whether failures are catastrophic (file not retrieved) or marginal (file ranked too low).

¹An earlier harness version averaged MRR/MAP only over reports reaching reranking with a resolvable ground truth, while Top- K was averaged over all reports. Table 6 and all downstream analyses now recompute MRR/MAP with *all* reports in the denominator (an unretrieved ground truth scores 0, consistent with the definitions here; Top- K was already computed this way). This lowers TRANSP-CNN's MRR/MAP markedly (CP-transfer MRR 0.380 $\rightarrow$ 0.214) and BLAZE's marginally (0.392 $\rightarrow$ 0.388 at the time); COOBA is unaffected. Per-bug ranks for all 756 runs are archived, so no retraining was needed. BLAZE's numbers were subsequently revised again to 0.298 by a separate, later correction (restricting its candidate pool to the shared FAISS top-300, Section 5.4); the 0.388 figure reflects only the denominator fix in isolation.

3.8. Domain Gap Measurement

To quantify structural dissimilarity between source and target, a **domain gap score** is computed for every pair by training a binary logistic regression classifier (source files labeled 0, target labeled 1, on file-level BGE embeddings) on a balanced 80/20 stratified split (ℓ_2 regularization, default $C = 1.0$, up to 1000 iterations); the held-out test accuracy is the score. A score of 0.5 means the two projects are indistinguishable in embedding space (no domain shift); 1.0 means fully disjoint regions (maximum shift). This is grounded in $\mathcal{H}$ -divergence theory (Ben-David et al., 2010), which formalizes domain separability as the accuracy of the best distinguishing classifier; the linear probe here is a practical instantiation, avoiding a more powerful classifier's risk of overfitting on small embedding datasets, and has separately been widely used as a separability diagnostic in the domain-adversarial literature (Ganin et al., 2016). Scores are computed once per pair and shared across all three model evaluations.

Across the 63 pairs, scores are tightly concentrated at the high end (min 0.976, median 0.989, max 0.997, mean 0.988, SD 0.006). Essentially every pair is near-perfectly separable regardless of shared domain. A logistic probe can separate almost any two distinct codebases this well, so this restricted range should be kept in mind when interpreting the null correlation between domain gap and CPL gain (Section 4): a null result here is at least as consistent with too little variance to detect an effect as with domain distance genuinely not mattering.

3.9. Statistical Analysis Methods

The analysis below follows recent guidance on statistical practice in empirical software engineering, favoring non-parametric tests, explicit effect-size reporting, and equivalence testing over reliance on unadjusted p -values alone (Gomes de Oliveira Neto et al., 2019).

RQ1, Wilcoxon Signed-Rank Test. To assess whether CP-transfer outperforms WP-small, a **Wilcoxon signed-rank test** (Wilcoxon, 1945; Arcuri and Briand, 2011) is applied per metric: observations are naturally paired (one CPT and one WPS result per pair under identical conditions), and improvement distributions cannot be assumed normal across so diverse a corpus. A one-sided alternative is tested (H_1 : CPT > WPS). Effect sizes are reported as Cohen's d (Cohen, 1988; Kampenes et al., 2007; Arcuri and Briand,

2014) (≥ 0.8 large, ≥ 0.5 medium, ≥ 0.2 small) for comparability with prior literature, alongside the matched-pairs rank-biserial correlation r_{rb} , the signed-rank-aligned effect size free of normality assumptions.

Significance markers denote unadjusted p -values ($\star\star\star p < 0.001$, $\star\star p < 0.01$, $\star p < 0.05$, n.s. $p \geq 0.05$); with five metrics tested per model family, the Bonferroni-adjusted threshold is $\alpha/5 = 0.010$, and the text states explicitly which comparisons survive it.

Equivalence Testing (TOST). Equivalence claims cannot rest on a non-significant difference test, since absence of evidence is not evidence of absence. Where the paper reports equivalence, we apply **two one-sided tests (TOST)** (Schuirmann, 1987; Lakens, 2017), implemented as Wilcoxon tests against null hypotheses shifted by $\pm\delta$, with margin $\delta = 0.05$ MRR. This margin is motivated, not derived: at the viability threshold MRR = 0.20 (ground truth around rank five), 0.05 MRR is exactly the reciprocal-rank gap between rank 4 (1/4) and rank 5 (1/5) for a single bug, defensible but not uniquely correct, since MRR is a population average rather than a per-bug statistic, so we report which conclusions are sensitive to it at $\delta = 0.02$ and 0.10 (Section 4 notes where the conclusion changes). Equivalence is declared when both one-sided tests reject at $\alpha = 0.05$.

RQ2, Spearman Rank Correlation. Spearman's ρ (Spearman, 1904) is computed between each project-level feature and CPT-WPS gain, preferred over Pearson's r since the corpus spans several orders of magnitude in codebase size and bug count, making linear association inappropriate.

RQ3, Source Selection Evaluation. Kendall's τ (Kendall, 1938) is computed between each heuristic's source ordering and the oracle ordering (by actual CP-transfer MRR) for targets with at least three candidate sources. **Hit@1** is the fraction of targets where the heuristic picks the oracle-best source; mean % of oracle performance achieved is also reported.

Statistical Power and Sample Size. With $n = 63$, a one-sided Wilcoxon test achieves approximately 98% power to detect a medium effect ($d = 0.5$) at $\alpha = 0.05$, adequately powered for the medium-to-large effects observed for BLAZE and TRANP-CNN. Power was estimated via simulation under a normal approximation to the signed-rank statistic, 10^4 replications across a grid of effect sizes and sample sizes.

4. Results

4.1. RQ1: Does CPL Outperform Within-Project Training Under Label Scarcity?

4.1.1. Scenario Performance Overview

CP-transfer achieves MRR = 0.298 for BLAZE versus WP-small's 0.209, a mean gain of +0.089 (43% relative). The ordering WP-large > CP-transfer > CP-cold-start $\approx$

WP-small holds on three of five metrics (MRR, Top-5, Top-10); MAP is effectively tied (0.181 vs. 0.182), and on Top-1 cold-start (0.126) trails WP-small (0.136), the one metric where it does not lead. In practical terms, a CP-transfer team can expect to rank the correct file in the top 5 in roughly 40% of queries (Top-5 = 0.395) versus 28% without transfer, a concrete triage-workflow improvement.

For TRANP-CNN, CP-transfer achieves MRR = 0.214 versus WP-small's 0.185, a +0.029 gain (16% relative), and is *statistically equivalent to WP-large* (0.219): a TOST test bounds the difference within ± 0.05 MRR ($p < 0.001$; two-sided Wilcoxon $p = 0.57$, 49.2% win rate), holding even at the tightest tested margin ($\delta = 0.02$, $p < 0.001$). TRANP-CNN is the only model here for which 20% target fine-tuning reaches its own full within-project performance.

For COOBA, the gain is +0.018 MRR (13% relative), smaller in both absolute and relative terms, and cold-start collapses to near-zero (MRR = 0.027). Zero-shot transfer fails without target-side signal for this architecture. Table 6 details the full per-scenario means.

4.1.2. Statistical Significance and Effect Size

Table 7 presents Wilcoxon signed-rank test results and Cohen's d for the CP-transfer versus WP-small comparison across all metrics and models.

For BLAZE, cross-project transfer significantly outperforms within-project limited training across all five metrics ($p < 0.001$, $\star\star\star$). Win rates range from 79.4% (Top-10) to 96.8% (MAP), and effect sizes are large by Cohen's convention throughout ($d = 0.84$ – 1.17 ; $r_{rb} = 0.91$ – 0.97), large, though part of this gain may reflect CP-transfer's larger target-side share (Section 5.4).

For TRANP-CNN, the advantage is also significant across all five metrics (win rates 47.6%–71.4%; $d = 0.32$ – 0.66 , $r_{rb} = 0.37$ – 0.66 ; $p < 0.001$ on MRR/MAP/Top-5, $p < 0.01$ on Top-1, $p < 0.05$ on Top-10, the only metric not surviving the Bonferroni threshold of 0.010), placing it alongside BLAZE as significant and meaningful on four of five metrics. A medium effect is statistically detectable at this sample size, but is estimated from a single seed per pair, and the run-to-run spread reaches 0.25 MRR (Section 5.4), so individual-pair reproducibility should not be assumed from the aggregate effect size.

For COOBA, the advantage is present but marginal: nominally significant on all five ($p = 0.024$ – 0.048 , $\star$) but none survives Bonferroni; win rates (42.9%–60.3%) are close to chance on several metrics, and effect sizes are small ($d = 0.21$ – 0.30 ; $r_{rb} = 0.24$ – 0.39), detectable at 63 pairs but too small for consistent per-deployment benefit.

We classify a model as exhibiting *practically beneficial* CPL if it meets both: significance on MRR/MAP at the Bonferroni threshold *and* at least a medium effect ($d \geq 0.5$, the conventional minimum for consistent individual-deployment benefit; a small effect is detectable at $n = 63$ but too small to reliably manifest in any one pair). This is a post hoc criterion, chosen after observing the effect sizes above ($d = 1.16/0.65/0.21$ – 0.30), not a pre-registered threshold,

Table 6

Mean bug localization performance across source→target pairs per model and scenario. WP-small is the primary baseline; CP-transfer is the primary CPL condition. Bold denotes the best CPL scenario per model. FAISS (retrieval stage) shows mean cosine-similarity ranking before any model reranking, applicable to all three models. All values on the same held-out test set (20% of target bugs, seed 42). MRR and MAP are averaged over *all* held-out test reports (unretrieved ground truth scores 0); see the footnote in Section 3.7. Top- K is unchanged.

Model	Scenario	MRR	MAP	Top-1	Top-5	Top-10
BLAZE ($n = 63$)	FAISS retrieval	0.029	0.029	0.003	0.019	0.072
	WP-small	0.209	0.182	0.136	0.282	0.362
	WP-large	0.344	0.302	0.258	0.450	0.508
	CP-cold-start	0.210	0.181	0.126	0.295	0.378
	CP-transfer	0.298	0.257	0.213	0.395	0.468
COOBA ($n = 63$)	FAISS retrieval	0.029	0.029	0.003	0.019	0.072
	WP-small	0.138	0.120	0.057	0.216	0.313
	WP-large	0.192	0.165	0.112	0.280	0.370
	CP-cold-start	0.027	0.023	0.008	0.035	0.059
	CP-transfer	0.156	0.134	0.075	0.237	0.339
TRANP-CNN ($n = 63$)	FAISS retrieval	0.029	0.029	0.003	0.019	0.072
	WP-small	0.185	0.159	0.107	0.274	0.351
	WP-large	0.219	0.192	0.132	0.314	0.387
	CP-cold-start	0.038	0.033	0.010	0.055	0.095
	CP-transfer	0.214	0.185	0.134	0.306	0.370

and happens to separate BLAZE and TRANP-CNN cleanly from COOBA: BLAZE ($d = 1.16$) and TRANP-CNN ($d = 0.65$) meet both criteria, COOBA ($d = 0.27$, no metric surviving correction) meets neither.

Taken together, the three models span a wide range of CPL benefit under identical conditions. That two architecturally distinct models both show benefit suggests the result is not an artifact of one model's inductive bias, though with one model per paradigm this is not a methodological replication. The three-tier effect-size profile (1.16/0.65/0.27) maps to deployment risk: BLAZE and TRANP-CNN showed consistent benefit in this corpus, while COOBA requires empirical validation before adoption.

4.1.3. Win/Loss Asymmetry

Whether CPL helps on average matters less than *what happens when it does not*: a model winning 60% of the time but losing catastrophically in the rest is worse than one winning 80% with negligible losses. For BLAZE, wins are wide (median +0.079 MRR, $n = 60$) and the 3 losing pairs are marginal (median −0.009, $n = 3$), smaller than the 0.05–0.07 MRR run-to-run spread (Section 5.4), so these losses read as measurement noise rather than a precise gain-to-loss ratio.

For TRANP-CNN, wins also exceed losses (median +0.036, $n = 43$; median −0.009, $n = 20$) but with meaningfully higher loss exposure than BLAZE, though still below the noise floor. Practitioners should expect roughly one in three deployments to underperform WP-small under this strict (any-negative-gain) definition, with larger gains in the rest.

For COOBA, gains and losses are closer in magnitude (median +0.032, $n = 38$; median −0.022, $n = 25$), a near-symmetric profile where losses can rival gains in nearly a third of deployments, calling for empirical validation before adoption. This loss-exposure ordering (BLAZE lowest, then TRANP-CNN, then COOBA) mirrors the effect-size ordering, though the BLAZE/TRANP-CNN magnitudes should be read against the noise floor above rather than as precise estimates. Figure 2 illustrates the gain distributions.

4.1.4. Zero-Label Deployment: The Cold-Start Finding

Cold-start CPL (a source-trained model with zero target annotations) removes the annotation bottleneck entirely, making its viability central to CPL's utility case for data-scarce teams.

For BLAZE, CP-cold-start (MRR = 0.210) is indistinguishable from WP-small (0.209) on any of five metrics (two-sided Wilcoxon, all $p \geq 0.48$), and TOST establishes equivalence within ± 0.05 MRR ($p = 0.0001$; sensitive to margin: not established at $\delta = 0.02$, $p = 0.070$; strengthens at $\delta = 0.10$, $p < 0.001$). This should be read specifically as *a BGE-based reranker deployed without target labels performs comparably to WP-small*, not as a claim about cross-project transfer in general: BLAZE re-scores its own retrieval signal (with better MaxSim aggregation) rather than applying an independent signal to a pool it never sees, unlike TRANP-CNN and COOBA (Section 5.4 discusses this asymmetry).

Figure 3's cold-start viability heatmap shows 46.0% of pairs (29/63) reaching MRR ≥ 0.20 , ground truth around rank five, a triage-inspectable shortlist, so annotation effort

Table 7

Wilcoxon signed-rank test results, Cohen's d with 95% CI, and matched-pairs rank-biserial correlation r_{rb} for CP-transfer vs. WP-small (one-sided, H_1 : CPT > WPS). **Win % here uses a strict zero threshold** (CPT > WPS MRR, any positive margin counts as a win); this is a different, more permissive threshold than the ± 0.01 MRR neutral band used for the negative-transfer rate and for the within/cross-domain win rate in Table 9, so the two should not be compared directly. Stars denote unadjusted p -values: $***$ ($p < 0.001$), $**$ ($p < 0.01$), $*$ ($p < 0.05$), n.s. The Bonferroni-adjusted threshold within each 5-metric family is $\alpha/5 = 0.010$; all BLAZE metrics and TRANP-CNN MRR/MAP/Top-1/Top-5 survive it, while all COOBA metrics and TRANP-CNN Top-10 do not. Note also that the Bonferroni family here treats the five metrics as independent tests, though MRR and MAP are near duplicates and the Top- K values are nested, so this correction is conservative.

Model	Metric	n	Win %	Δ (mean)	Δ (median)	Cohen's d	95% CI (d)	r_{rb}	Sig.
BLAZE	MRR	63	95.2%	+0.089	+0.074	1.16	[+0.96, +1.46]	0.96	***
	MAP	63	96.8%	+0.075	+0.060	1.17	[+0.97, +1.46]	0.97	***
	Top-1	63	87.3%	+0.077	+0.069	1.09	[+0.86, +1.41]	0.91	***
	Top-5	63	84.1%	+0.113	+0.077	1.00	[+0.83, +1.25]	0.97	***
	Top-10	63	79.4%	+0.106	+0.071	0.84	[+0.69, +1.06]	0.91	***
COOBA	MRR	63	60.3%	+0.018	+0.003	0.27	[+0.04, +0.50]	0.26	*
	MAP	63	60.3%	+0.014	+0.007	0.24	[−0.00, +0.47] [†]	0.24	* [†] For
	Top-1	63	42.9%	+0.018	+0.000	0.30	[+0.08, +0.52]	0.39	*
	Top-5	63	55.6%	+0.021	+0.011	0.21	[−0.04, +0.46]	0.25	*
	Top-10	63	55.6%	+0.027	+0.009	0.29	[+0.07, +0.49]	0.28	*
TRANP-CNN	MRR	63	68.3%	+0.029	+0.021	0.65	[+0.45, +0.87]	0.62	***
	MAP	63	71.4%	+0.026	+0.018	0.66	[+0.47, +0.86]	0.66	***
	Top-1	63	54.0%	+0.027	+0.011	0.45	[+0.23, +0.66]	0.46	**
	Top-5	63	60.3%	+0.033	+0.024	0.53	[+0.34, +0.74]	0.58	***
	Top-10	63	47.6%	+0.019	+0.000	0.32	[+0.16, +0.48]	0.37	*

COOBA MAP, the Wilcoxon p -value ($*$) and the parametric Cohen's d CI are derived from different distributional assumptions; a CI that marginally crosses zero is consistent with a significant non-parametric test at $n = 63$. Top- K metrics are integer-valued and produce exact ties (equal Top- K score) at a rate the continuous MRR/MAP metrics do not: COOBA Top-1 has 19 tied pairs (27 wins, 17 losses) and TRANP-CNN Top-10 has 14 tied pairs (30 wins, 19 losses); reported Win % and median $\Delta = +0.000$ for these rows reflect a sign test over win/loss/tie counts, not an error.

can be eliminated for roughly half of scenarios with a well-matched source, though the majority case now needs some target-side signal. Once a modest label set is added, within-domain CP-transfer raises this further: BLAZE beats WP-small in 90.0% of within-domain pairs (Table 9).

COOBA collapses to near-zero in cold-start (MRR = 0.027, matching the raw FAISS baseline of 0.029, far below WP-small's 0.138, $p < 0.001$). TRANP-CNN also fails, though less severely: 0.038 versus WP-small's 0.185 ($p < 0.001$), 1.3 $\times$ the FAISS baseline, some residual utility from source-trained weights, but only 21% of WP-small's performance. Both scalar-ranking architectures need target-side calibration their scoring functions remain anchored to the source distribution without it. The dichotomy is consistent with an architectural account: BLAZE's embedding similarity carries over to unseen targets on average, while TRANP-CNN's and COOBA's learned scores do not.

RQ1 Summary

CPL benefit under label scarcity ranges from large to none surviving correction.

- **BLAZE:** $d = 1.16$, win rate 95.2%, $p < 0.001$; cold-start equivalent to WP-small (TOST, $p = 0.0001$), viable in 46.0% of pairs; losses rare and small.
- **TRANP-CNN:** $d = 0.65$, win rate 68.3%, $p < 0.001$; CP-transfer equivalent to WP-large (TOST, $p < 0.001$).
- **COOBA:** $d = 0.27$, no metric survives Bonferroni correction; cold-start fails.

4.2. RQ2: What Project-Level Characteristics Determine CPL Success or Failure?

4.2.1. Target Codebase Size

Target codebase size is the clearest size-stratified moderator of CPL benefit here. Table 8 presents CPL gain stratified by target size.

For BLAZE, gain is largest for small targets (mean Δ MRR = +0.112, <100K LoC) but stays meaningful throughout (+0.081 medium, +0.062 large). CPL remains a viable choice even for large, complex codebases. For

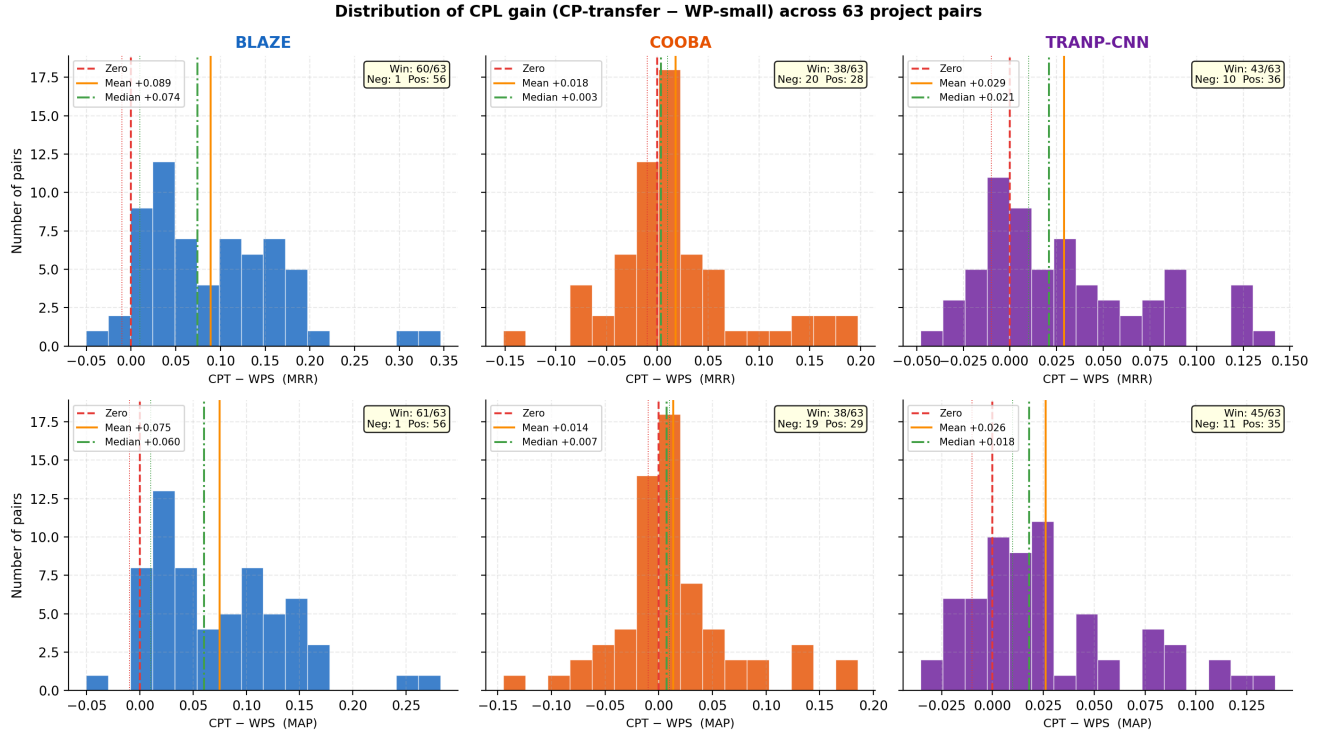


Figure 2: Distribution of CP-transfer minus WP-small MRR gain per source→target pair ($n = 63$ pairs each, all three models). Positive values indicate CPL benefit. BLAZE (blue) shows a pronounced rightward skew, TRANP-CNN (purple) moderate asymmetry with a wider spread, and COOBA (orange) a near-symmetric distribution (see text for medians). The vertical dashed line marks zero; dotted lines mark the ± 0.01 MRR threshold used to classify positive/negative transfer.

TRANP-CNN, CP-transfer narrowly *exceeds* WP-large for small targets (0.351 vs. 0.345), the only size-group aggregate here where cross-project transfer surpasses the within-project ceiling (individual pairs also cross this line; Section 4.4); medium and large targets show modest gains (+0.017, +0.021). For COOBA, target LoC acts as a near-hard ceiling: gains of +0.025 (small) fall to +0.017 (medium) and +0.006 (large, smaller than measurement noise). CPL is only worth attempting for COOBA on small targets, with no measurable benefit on large ones despite the added source signal.

These gains warrant caution: FAISS recall at $K = 300$ falls below 50% for the three largest-codebase projects (ansible 31.3%, wagtail 34.6%, meson 45.5%), so part of the apparent LoC-driven degradation reflects a shared retrieval ceiling rather than architecture alone. The within-model comparison (CPT vs. WPS) is unaffected, since both face the same constraint, but absolute gain magnitudes for large targets are suggestive rather than definitive.

Figure 4 shows the distribution of CPL gain by target size group. For BLAZE, the gain distributions for all size groups are centered positively. For COOBA, the medium and large groups are centered at or below zero. For TRANP-CNN, all groups are positive, with the small group exceeding the WP-large ceiling.

4.2.2. Negative Transfer

Negative transfer, where source-project knowledge actively harms target performance, is CPL's central risk, determining whether it is a safe default or requires careful source selection. We define it as CP-transfer MRR < WP-small MRR by more than 0.01 ($|\Delta| \leq 0.01$ treated as no-effect).

For BLAZE, only 1 of 63 pairs (1.6%) exhibits negative transfer: qiskit → numpy (−0.050 MRR, cross-domain, 277K-LoC target), a loss at the edge of the 0.05–0.07 run-to-run variance floor (Section 5.4). For TRANP-CNN, 10 of 63 (15.9%, roughly one in six) are negative, worst at ansible → prefect (−0.048), lightning → prefect (−0.031), and localstack → lightning (−0.030), concentrated where high-specificity codebases (prefect, lightning) are targets. For COOBA, 20 of 63 (31.7%, nearly a third) are negative, worst at meson → jupyterlab (−0.152), qiskit → jupyterlab (−0.084), and ansible → jupyterlab (−0.081); jupyterlab's recurrence as most-harmed target is consistent with (though does not establish) COOBA's AST-based representations overfitting to source structure that conflicts with jupyterlab's frontend-heavy codebase, which we did not verify directly. Practitioners should treat TRANP-CNN and COOBA CPL as requiring empirical validation per deployment, not a universally safe upgrade. The three-model spectrum (1.6%→15.9%→31.7%) aligns with the win/loss asymmetry ordering and gives a consistent per-architecture risk profile.

Table 8

CP-transfer vs. WP-small MRR gain by target codebase size group. Δ = CP-transfer MRR – WP-small MRR. WP-large MRR is shown for all models; the dagger marks the only size-group aggregate where mean CP-transfer exceeds mean WP-large.

Model	Size Group	n	WP-small MRR	WP-large MRR	CP-transfer MRR	Δ MRR
BLAZE	Small (<100K LoC)	25	0.189	0.348	0.301	+0.112
	Medium (100K–300K)	24	0.242	0.376	0.323	+0.081
	Large (>300K LoC)	14	0.188	0.284	0.250	+0.062
COOBA	Small (<100K LoC)	25	0.219	0.294	0.243	+0.025
	Medium (100K–300K)	24	0.099	0.152	0.116	+0.017
	Large (>300K LoC)	14	0.062	0.080	0.068	+0.006
TRANP-CNN	Small (<100K LoC)	25	0.306	0.345	0.351[†]	+0.045
	Medium (100K–300K)	24	0.125	0.162	0.141	+0.017
	Large (>300K LoC)	14	0.072	0.090	0.093	+0.021

[†]CP-transfer exceeds WP-large for small targets (0.351 vs. 0.345), the only size-group aggregate in this study where cross-project transfer surpasses the within-project ceiling.

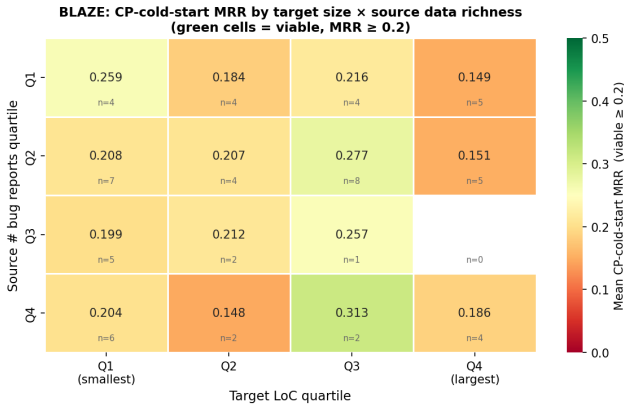


Figure 3: BLAZE cold-start viability heatmap (CP-cold-start MRR), binned by target LoC quartile (columns) and source bug-report-count quartile (rows), with cell counts n annotated. 9 of 15 populated cells exceed the MRR = 0.20 viability threshold; the six exceptions cluster at the extremes of target codebase size (source Q1/Q2/Q4 $\times$ target Q4, MRR = 0.149/0.151/0.186; source Q1/Q4 $\times$ target Q2, MRR = 0.184/0.148; source Q3 $\times$ target Q1, MRR = 0.199, just below threshold). One cell (source Q3 $\times$ target Q4) is empty ($n = 0$) and shown white. COOBA and TRANP-CNN are not shown: both achieve MRR < 0.20 almost everywhere in cold-start (0% and 1.6% of pairs respectively).

Figure 5 plots Spearman correlations between project-level features and CPL gain. *Target label volume*, setting WP-small’s absolute size (Section 3.6), is the strongest predictor, negatively: $\rho = -0.67$ for BLAZE, -0.61 for TRANP-CNN (both $p < 0.001$); target LoC, the source $\rightarrow$ target LoC ratio, and target verbosity are also significant for BLAZE ($\rho = -0.34, +0.35, +0.35$, all $p < 0.01$), though verbosity itself correlates with label volume ($\rho = -0.52$) and so is a related facet of the same under-resourced-target pattern rather than an independent fourth signal. The fewer labels a target already has, the more it gains from CPL; the same relationship is documented at the paradigm level (Section 5) recast per pair. Similarity-based features are

weaker: domain gap accuracy is non-significant for COOBA and TRANP-CNN ($|\rho| < 0.20$), and no measured feature predicts COOBA at all ($|\rho| < 0.20$, all $p > 0.10$); for BLAZE, domain gap accuracy is now also significant ($\rho = -0.29$, $p = 0.023$), a secondary signal not detectable pre-correction. So negative transfer for COOBA is unpredictable from these features; for BLAZE and TRANP-CNN, label volume is a useful if imperfect guide, and for BLAZE domain alignment adds modest further signal.

4.2.3. Source-Target Domain Alignment

Table 9 breaks CPL gain down by whether source and target share a domain (within-domain, Group A) or not (cross-domain, Groups B/C); Figure 6 shows the corresponding distributions. All within-domain pairs are Data Science/ML (Group A), so this contrast also reflects domain identity, not just alignment.

For BLAZE, within-domain pairs gain only slightly more (+0.124 MRR, 90.0% win rate) than cross-domain (+0.058, 87.9%), a narrow 2.1pp drop; cross-domain negative transfer remains rare (1 of 33). *Domain alignment has only a marginal effect for BLAZE*, and practitioners can expect cross-domain win rates close to within-domain. For TRANP-CNN, the effect is pronounced, within-domain 70.0% vs. cross-domain 45.5%, a 24.5pp drop (the steepest of the three, falling below the 50% threshold), indicating domain gap is a primary risk factor; within-domain sources should be prioritized. For COOBA, the effect is negligible (within-domain 43.3% vs. cross-domain 45.5%, -2 pp, cross-domain slightly higher), contradicting the naive expectation that adversarial adaptation helps most within-domain; this is consistent with COOBA’s outcome being driven by fit to source structure rather than domain proximity, making domain alignment an unreliable guide for its source selection.

Group B (14 of 33 cross-domain pairs) is built entirely around jupyterlab as the pivot; excluding it, cross-domain win rates become 84.2% (BLAZE), 47.4% (COOBA), 31.6% (TRANP-CNN), so the domain effect is not an artifact of the jupyterlab pivot for any model.

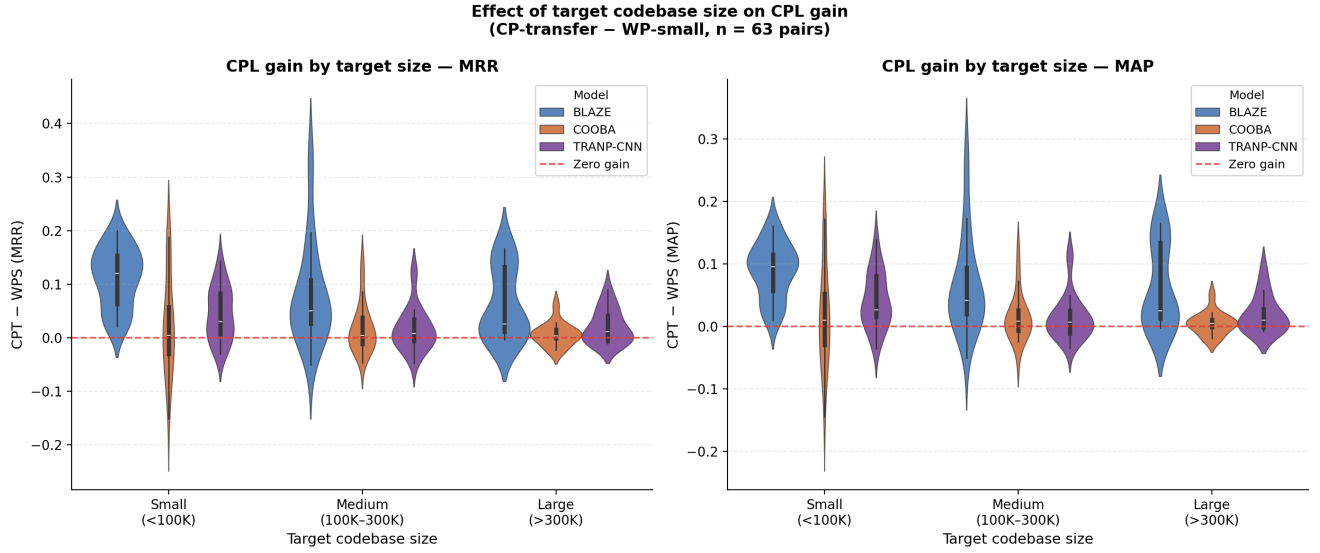


Figure 4: Distribution of CP-transfer minus WP-small MRR gain by target codebase size group (Small <100K, Medium 100K–300K, Large >300K LoC). Each violin shows the spread of pair-level gains for a given model (color) and size group; see Table 8 for the corresponding means.

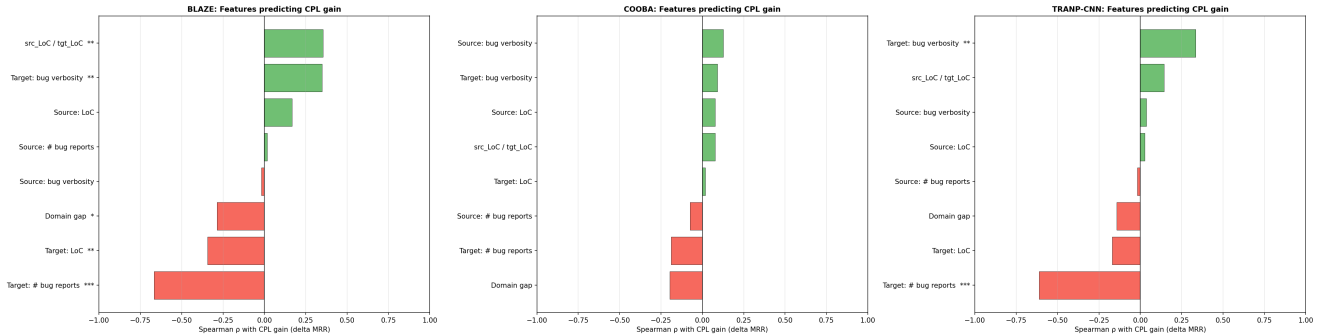


Figure 5: Spearman correlations between project-level metadata features (target and source LoC, target and source bug-report count, LoC ratio, domain gap accuracy, target and source bug-report verbosity) and CP-transfer minus WP-small MRR gain, shown for all three models (BLAZE, COOBA, TRANP-CNN); see text for which correlations reach significance.

The domain gap metric (logistic regression accuracy on file embeddings, Section 3.8) shows no significant correlation with gain for COOBA ($|\rho| < 0.12$, $p > 0.20$) or TRANP-CNN ($\rho = 0.11$, $p = 0.39$); for BLAZE it is now significant ($\rho = -0.29$, $p = 0.023$), emerging only after the FAISS-restriction correction. The domain binary is a stronger signal than this continuous metric for TRANP-CNN, a non-signal for COOBA, and for BLAZE the two signals now point the same direction.

4.2.4. Commutativity: Does Transfer Direction Matter?

For 28 symmetric pairs ($A \rightarrow B$ and $B \rightarrow A$ both evaluated), we test whether the smaller-LoC target consistently gains more, as the LoC-stratified results would predict. BLAZE and TRANP-CNN both show this in 57.1% of pairs (16/28), only modestly above the 50% random baseline, weak rather than strong evidence, so LoC asymmetry is not the dominant factor for either learned-ranker model; domain

alignment and bug distribution contribute more. COOBA is highest at 60.7% (17/28), broadly consistent with its small-target advantage, but the margin over the other two is narrow enough that this reads as a shared, mild tendency across all three architectures rather than a COOBA-specific signature.

Table 9

CPL gain by domain alignment (within-domain = same application category; cross-domain = different category). Δ MRR = mean CP-transfer – WP-small MRR. Win Rate uses the ± 0.01 MRR neutral band, a different (stricter) threshold than Table 7's zero-cutoff Win % (see that table's caption). TRANP-CNN shows the steepest domain sensitivity (24.5 percentage-point win-rate drop); COOBA shows essentially no domain effect (–2pp, cross-domain slightly higher); BLAZE shows only a marginal domain effect (2.1pp drop) and remains strong cross-domain (87.9% win rate).

Model	Domain Type	n	Δ MRR (mean)	Win Rate
BLAZE	Within-domain (DS×DS)	30	+0.124	90.0%
	Cross-domain	33	+0.058	87.9%
COOBA	Within-domain (DS×DS)	30	+0.027	43.3%
	Cross-domain	33	+0.010	45.5%
TRANP-CNN	Within-domain (DS×DS)	30	+0.042	70.0%
	Cross-domain	33	+0.017	45.5%

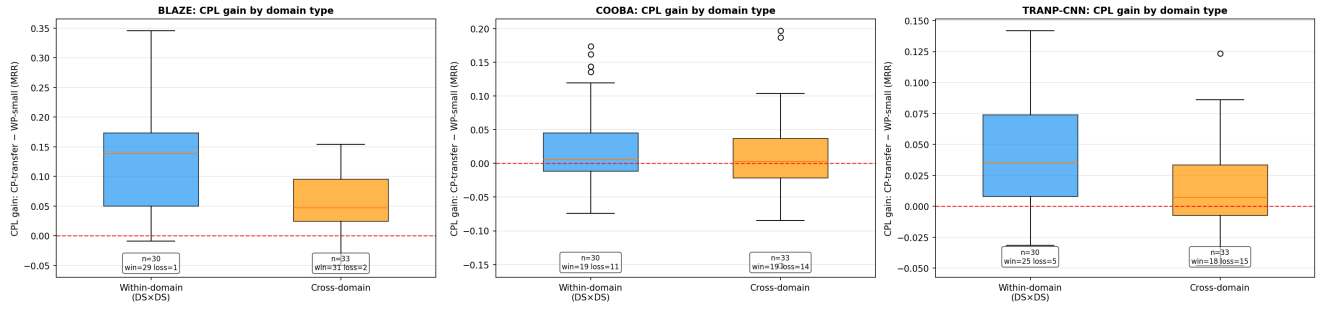


Figure 6: CPL gain distributions (CP-transfer – WP-small MRR) for within-domain (DS×DS) versus cross-domain pairs across all three models; see Table 9 for the corresponding win rates.

RQ2 Summary

CPL benefit is largest for small-codebase, within-domain targets, but neither is a prerequisite for BLAZE; the three models exhibit a consistent risk spectrum:

- **BLAZE:** 1.6% negative transfer, 2.1pp cross-domain sensitivity, 87.9% cross-domain win rate.
- **TRANP-CNN:** 15.9% negative transfer, 24.5pp cross-domain sensitivity (steepest), CPT exceeds WPL for small targets.
- **COOBA:** 31.7% negative transfer, LoC-sensitive, no domain sensitivity (–2pp).

Target label volume best predicts gain for BLAZE and TRANP-CNN ($\rho = -0.67, -0.61$); no measured feature predicts COOBA's outcomes.

4.3. RQ3: How Consistent Are CPL Outcomes Across Model Implementations?

4.3.1. Cross-Model Direction Agreement

Cross-model agreement tests whether the CPL benefit observed for one architecture predicts benefit for another on the same pair: if CPL were a property of the pair, all models should agree; if architecture-mediated, they should diverge. Table 10 reports pairwise direction agreement across all three model combinations.

Table 10

Pairwise cross-model direction agreement on CPL gain (CPT–WPS MRR, $n = 63$). Agreement = fraction of pairs where both models show the same directional outcome (positive, neutral, or negative gain). Spearman ρ measures the continuous correlation of pair-level CPL gains. ***: $p < 0.001$.

Model Pair	Agreement	Spearman ρ
BLAZE vs. COOBA	48%	0.150 (n.s.)
BLAZE vs. TRANP-CNN	62%	0.516***
COOBA vs. TRANP-CNN	44%	0.216 (n.s.)

BLAZE and TRANP-CNN agree in 62% of pairs with a moderate positive correlation ($\rho = 0.516, p < 0.001$), consistent with these two learned-ranker models sharing more CPL sensitivity than either shares with COOBA, which agrees with BLAZE in 48% and TRANP-CNN in 44%, neither significant (BLAZE vs. COOBA $p = 0.24$, COOBA vs. TRANP-CNN $p = 0.09$). The most pronounced divergences: BLAZE gains +0.136 MRR on meson→jupyterlab while COOBA loses –0.152 (COOBA's single worst outcome across all 63 pairs), and BLAZE gains +0.120 on qiskit→jupyterlab while COOBA loses –0.084. Figure 7 shows the cross-model scatter plots.

This has a natural interpretation: BLAZE and TRANP-CNN both assign relevance scores at inference time (cosine similarity vs. CNN scalar) with cross-project knowledge

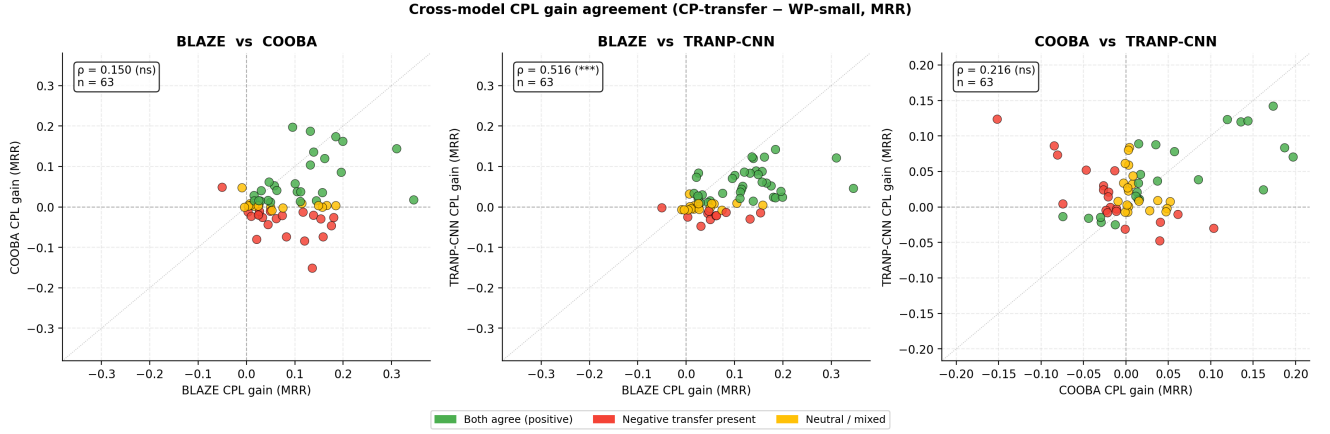


Figure 7: Cross-model scatter of CPT–WPS MRR gain across 63 shared source→target pairs, for all three model pairs: BLAZE vs. COOBA (left), BLAZE vs. TRANSP-CNN (center), and COOBA vs. TRANSP-CNN (right); see Table 10 for the corresponding correlations.

injected at training time, while COOBA’s adversarial mechanism trains a discriminator to erase source-target distinctions, a fundamentally different adaptation strategy producing different sensitivity to the same pairs. CPL outcomes therefore vary substantially across architectures on the same pair, which Section 5.2 interprets via the model’s adaptation mechanism: a practitioner should not assume one model’s outcome predicts another’s, and the 44%–62% agreement range is consistent with architecture being at least as strong a determinant of outcomes as the pair itself.

4.3.2. Source Quality as a Transfer Predictor

A natural source-selection heuristic is to pick the source that performs best within-project. We test this via Spearman ρ between the source’s WP-large MRR and its achieved CP-transfer MRR. For BLAZE, $\rho = -0.00$ ($p = 0.999$): trivially zero predictive power, so practitioners can select sources freely without within-project evaluation. For TRANSP-CNN, $\rho = -0.208$ ($p = 0.102$): negative but not significant, the same practical conclusion as BLAZE. For COOBA, $\rho = -0.296$ ($p = 0.019$): a significant *negative* predictor, sources on which COOBA performs best within-project actually transfer worse, conjectured (not verified by inspecting AST structure) to reflect overfitting to source-specific structural patterns that become a liability on the target. COOBA practitioners should therefore avoid within-project source performance as a selection criterion entirely.

All three models thus agree: source selection based on within-project quality is unreliable or counter-productive across all three paradigms.

4.3.3. Source Selection Heuristics

Table 11 evaluates two source-selection heuristics against the oracle, across targets with at least three candidate sources: *most-bugs* (largest bug corpus) and *domain-gap* (lowest embedding-space gap). Hit@1 is how often a heuristic picks the oracle-best source, Kendall τ the quality of its full ranking, and % oracle the achieved CP-transfer MRR relative to the best possible source.

For BLAZE, *most-bugs* gets Hit@1 = 23.1%, $\tau = -0.051$ (near-zero); *domain-gap* is worse on Hit@1 (15.4%) but weakly positive on τ (+0.020). Neither ranks the full list reliably, yet BLAZE reaches 89.3%–92.2% of oracle with either. Source selection has low stakes here; even a poorly chosen source is near-oracle. For TRANSP-CNN, *most-bugs* gets Hit@1 = 30.8%, $\tau = +0.110$ (the only positive τ of any heuristic across all three models); *domain-gap* gets Hit@1 = 23.1%, $\tau = -0.080$; oracle recovery is 91.1%–92.6% either way, though the 15.9% negative-transfer risk still makes within-domain validation advisable. For COOBA, *most-bugs* gets Hit@1 = 38.5%, $\tau = -0.194$; *domain-gap* gets Hit@1 = 23.1%, $\tau \approx 0$ (−0.007, expected: domain similarity carries no signal when the CPL mechanism itself erases domain-specific features); oracle recovery is uneven, 73.8% for *most-bugs*, 84.8% for *domain-gap*, the widest heuristic gap and, at its low end, the lowest oracle recovery in this study.

The takeaway is consistent across heuristics and models (Figure 8 visualizes *most-bugs* ranking quality): no heuristic reliably ranks sources ($\tau < 0.2$ throughout), yet all three models reach 74–93% of oracle with any of them, so sophisticated source selection is unnecessary for BLAZE or TRANSP-CNN. COOBA’s lower absolute recovery makes selection matter more, but neither heuristic is reliable there either. Empirical holdout validation on a small target sample remains the safest strategy.

Table 11

Source selection heuristic evaluation. Two heuristics tested: *most-bugs* (source with largest bug corpus) and *domain-gap* (source with lowest embedding-space domain gap). “% oracle” = mean CP-transfer MRR of selected source as a fraction of oracle CP-transfer MRR. No heuristic reliably ranks sources ($\tau < 0.2$ in all cases), yet oracle recovery remains high (74–93%) for all strategies.

Model	Strategy	Hit@1	Kendall τ	% Oracle
BLAZE	Oracle (upper bound)	100%	N/A	100%
	Most-bugs heuristic	23.1%	−0.051	89.3%
	Domain-gap heuristic	15.4%	+0.020	92.2%
COOBA	Oracle (upper bound)	100%	N/A	100%
	Most-bugs heuristic	38.5%	−0.194	73.8%
	Domain-gap heuristic	23.1%	−0.007	84.8%
TRANP-CNN	Oracle (upper bound)	100%	N/A	100%
	Most-bugs heuristic	30.8%	+0.110	92.6%
	Domain-gap heuristic	23.1%	−0.080	91.1%

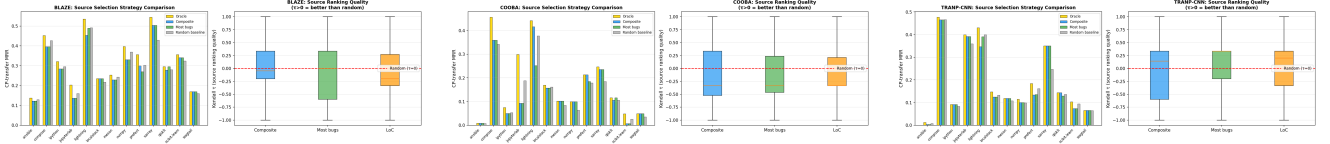


Figure 8: Source selection ranking quality (most-bugs heuristic) for BLAZE (left), COOBA (center), and TRANP-CNN (right). Bubble size proportional to target bug count; color indicates oracle rank of the selected source. Table 11 reports results for both heuristics.

RQ3 Summary

Cross-model agreement ranges from 44% (COOBA vs. TRANP-CNN) to 62% (BLAZE vs. TRANP-CNN), though BLAZE’s and TRANP-CNN’s gains are correlated ($\rho = 0.516$, $p < 0.001$). Source within-project quality does not predict transfer for BLAZE or TRANP-CNN, and is a significant *negative* predictor for COOBA ($\rho = -0.296$). All three models stay near-oracle under source selection (74–93% of oracle with any source), making heuristic source ranking low-priority relative to domain-gap validation.

4.4. Illustrative Cases

The aggregate findings above describe CPL behavior across 63 pairs and three models. Three project pairs make these patterns concrete: cold-start viability, architecture divergence on the same pair, and CP-transfer exceeding the ample-data ceiling.

Case 1: BLAZE cold-start viability (*lightning-ai* → *xarray*).

lightning-ai/lightning (51K LoC) sources *pydata/xarray* (142K LoC), a within-domain pair. BLAZE cold-start (CPC, MRR = 0.384) exceeds WPS (0.199) by +0.185, the source-trained model beats 10% annotated target-only training with zero target labels. TRANP-CNN and COOBA cold-start both collapse (0.009 and 0.028), far below their own WPS.

Model	WPS	WPL	CPC	CPT
BLAZE	0.199	0.551	0.384	0.545
TRANP-CNN	0.197	0.236	0.009	0.243
COOBA	0.086	0.182	0.028	0.103

For a team beginning xarray localization with no labeled data, a pre-trained BLAZE model sourced from lightning outperforms annotating 10% of local bugs. Neither other architecture replicates this: their score functions are calibrated to the source distribution and cannot rank target files without target-side fine-tuning.

Case 2: Architecture divergence on the same pair (wagtail → docker/compose). *wagtail/wagtail* (286K LoC) sources *docker/compose* (35K LoC), a cross-domain pair. BLAZE gains +0.083 MRR (CPT–WPS); COOBA experiences *negative transfer* (−0.074) on the identical target; TRANP-CNN is essentially flat (−0.014), already strong within-project.

Model	WPS	WPL	CPC	CPT
BLAZE	0.370	0.495	0.246	0.453
COOBA	0.279	0.450	0.084	0.205
TRANP-CNN	0.470	0.452	0.049	0.457

The same source simultaneously produces a genuine BLAZE gain and COOBA’s negative transfer on the identical target: a COOBA practitioner would call wagtail harmful, a BLAZE practitioner mildly helpful. This pair’s BLAZE WPS run (0.370) sits at the median of the three WPS runs recorded

for docker/compose, so the divergence is not a run-to-run artifact (Section 5.4); no model-agnostic viability judgment is possible here.

Case 3: TRANP-CNN CP-transfer exceeds full within-project training (jupyterlab → scikit-learn). *jupyterlab/jupyterlab* (39K LoC) sources *scikit-learn/scikit-learn* (376K LoC). With scikit-learn's WPS collapsed to 0.006, TRANP-CNN's CP-transfer (0.095) exceeds WP-large (0.039) by +0.056, joint training on the full source corpus plus 20% target labels still beats four times as much target-only data. BLAZE CPT (0.298) outperforms its WPS (0.154) but not its WPL (0.373); COOBA gains minimally.

Model	WPS	WPL	CPC	CPT
BLAZE	0.154	0.373	0.103	0.298
COOBA	0.016	0.036	0.005	0.031
TRANP-CNN	0.006	0.039	0.005	0.095

TRANP-CNN trains on jupyterlab's full corpus in the same run as just 20% of scikit-learn's bugs, and the resulting calibration surpasses four times as much local data alone (WPL), though both figures are small in absolute terms. This pair's WPL (0.039) sits below the large-target mean of 0.090 in Table 8, reflecting scikit-learn's domain specificity and the FAISS ceiling at this scale rather than a data anomaly, illustrating why CPL is especially valuable early in a project's life, before a meaningful labeled history has accumulated.

5. Discussion

5.1. Practical Implications

The results support six concrete recommendations for practitioners considering CPL deployment.

1. *For BLAZE, CPL showed consistent benefit with low loss exposure in this corpus.* CP-transfer outperformed WP-small in the large majority of deployments (Table 7), with losses rare and within the run-to-run noise floor (Section 4). *Zero-label deployment matches WP-small on average, though it is viable in under half of pairs.* Deployment can begin without target annotations, with labels accumulated subsequently for fine-tuning.

2. *Target codebase size moderates benefit; validate empirically for large targets.* CPL gain is largest for small-codebase targets and drops for medium and large targets, though the two converge rather than continuing to decline (Table 8); for COOBA, benefit essentially disappears beyond medium size. Practitioners with large-codebase targets and COOBA-style tools should validate CPL empirically before relying on it.

3. *Choose sources by domain, not by ranking heuristics or within-project quality.* No tested heuristic reliably ranks source projects for any model, yet all three achieve near-oracle CP-transfer performance regardless of source choice (Table 11), so source selection has low stakes for

BLAZE and TRANP-CNN. Domain category membership is the most actionable criterion, though the within-domain evidence here comes entirely from Data Science pairs: same-domain sources give a markedly higher win rate for TRANP-CNN, and any conveniently available within-domain source beats investing in ranking by corpus size or embedding proximity. Within-project quality should not be used as a selection criterion for any architecture; for COOBA it is a significant *negative* predictor (Section 4), conjectured to reflect overfitting to source-specific AST patterns, and COOBA's lowest oracle recovery and highest negative-transfer rate make holdout validation advisable there.

4. *With 20% target labels, TRANP-CNN reaches full within-project training performance.* With 20% of bugs labeled and a domain-aligned source, TRANP-CNN's CP-transfer performance is statistically equivalent to WP-large, and exceeds it for small-codebase targets (Section 4), the highest CPL ceiling *relative to its own baseline* of the three models, though BLAZE still leads on absolute MRR and Top-K throughout (Table 6). This carries a substantial negative-transfer risk concentrated in cross-domain pairs, so domain alignment should be validated before deploying TRANP-CNN cross-domain.

5. *Empirical holdout validation is the safest guard against negative transfer.* Target label volume predicts CPL gain for BLAZE and TRANP-CNN, but no measured feature reliably predicts pair-level gain for COOBA (Section 4). When deployment risk is high, the safest approach is a small annotated target holdout before committing to cross-project transfer, especially for COOBA (roughly one in three deployments negative) and, to a lesser extent, TRANP-CNN (roughly one in six); a project able to hold one out is, by definition, not fully in the zero-label regime this study is written for (Section 5.4 gives a rough usable size).

6. *CP-transfer's benefit has a compute cost that WP-small does not.* CP-transfer trains on the full source corpus plus a larger target-side share than WP-small, so its cost scales with source-corpus size rather than staying fixed (Section 5.4). BLAZE's zero-label cold-start deployment avoids this entirely, part of its appeal in recommendation 1; for TRANP-CNN, which needs the full CP-transfer run to reach its ceiling, this cost is worth weighing against WP-small alone when the source corpus is large and the expected gain modest.

5.2. Architectural Interpretation of CPL Profiles

CPL outcomes vary substantially across architectures on the same source→target pairs, and we offer a mechanistic account: models that rank at inference time via cosine similarity carry cross-project semantics in their representations, while models applying learned scalar scores need target-side calibration and cannot transfer without it. The three models span distinct paradigms of how a bug localization model evaluates candidates at inference time. The experimental design was structured to make architectural attribution possible

rather than merely correlational: all three are evaluated on identical datasets with identical train/test splits (seed 42), retrieve from the same BGE-based FAISS indexes, and are benchmarked across the same 63 pairs, with training data access, retrieval candidates, and pair selection held constant. Each model consumes its native input representation (BGE for BLAZE, learned tokens for TRANP-CNN, GloVe-300 text/AST for COOBA), treated as a constituent of the architecture under test. We interpret each model's **evaluation mechanism**, what its learned parameters and representations do to rank candidates at test time, as the dimension most consistent with the divergent profiles below. This controlled design lets performance differences be attributed to architectural choices rather than confounding conditions, though as Section 5.4 notes, representation quality is bundled into each architecture rather than independently controlled, so "evaluation mechanism" should be read as our best-supported interpretation rather than an isolated, independently-verified cause.

The divergent CPL profiles are consistent with that mechanism.

BLAZE (embedding-based dual-encoder). BLAZE scores candidates at inference time using cosine similarity between frozen BGE backbone embeddings, with no learned scalar weight applied at ranking time. The ordering is determined by the frozen pre-trained representations and the trained adapters, with no learned scalar at ranking time; the frozen backbone, built on a diverse multilingual code corpus, already carries cross-project semantics before fine-tuning begins. Cold-start viability follows: there is nothing project-specific to calibrate, since no target-specific ranking weights exist, and the small, rare losses observed (median -0.009 MRR, 3 of 63 pairs) follow from the same property. Similarity-based ranking is inherently smooth, so a suboptimal source produces suboptimal but not catastrophically wrong similarities (bounding loss), while well-aligned source-target spaces can yield large gains. Source selection therefore has low stakes: even a poorly chosen source produces only marginally off rankings.

Two competing hypotheses warrant consideration: (i) *representation quality* (BLAZE reranks with modern frozen BGE embeddings while TRANP-CNN and COOBA use their original designs', so its profile might reflect representation quality rather than mechanism), and (ii) *model age* (BLAZE, 2025, is four to six years newer, so the gap might reflect general progress). Three observations favor the evaluation-mechanism account, though not mutually exclusive and still an interpretation: if representation quality were the sole factor, cold-start should be near-optimal without adapter training, yet CP-cold-start (0.210) trails CP-transfer (0.298), so fine-tuning adds signal beyond the pre-trained representations; BLAZE's gain-to-loss asymmetry ($+0.079$ vs. -0.009 MRR) reflects ranking smoothness, not representation capacity, since a strong backbone alone would not guarantee bounded losses; and the two non-BGE rerankers differ from each other in profile ($d = 0.65$ vs. 0.27), so

representation vintage alone does not determine transfer behavior. Section 5.4 records the residual representation asymmetry as a construct threat.

COOBA (GNN with adversarial domain adaptation). COOBA trains a Gradient Reversal Layer discriminator to erase project-specific signals from the shared GNN feature space. This adversarial objective is the CPL mechanism, forcing the encoder toward domain-invariant features by preventing the discriminator from distinguishing source from target representations. But the mechanism needs target-side representations *during training* for the discriminator to contrast against the source distribution; with zero target data it gets no target signal, so the resulting features stay calibrated to the source AST vocabulary (identifier namespaces, call-graph structures, type hierarchies, all project-specific), which misalign on a new target, explaining the near-zero cold-start MRR (0.027 vs. 0.138 for WP-small). The counter-intuitive finding that higher source within-project performance predicts *worse* transfer follows the same logic: a model that has learned source AST patterns very precisely has encoded more source-specific features that generalize less to a new project's vocabulary.

TRANP-CNN (CNN reranker). TRANP-CNN produces a scalar relevance score per (bug report, candidate file) pair via parallel convolutional feature extractors and margin ranking loss, calibrated during training to the source project's ranking difficulty. Without target examples this score stays anchored to the source distribution, its scale and decision boundary not transferring, explaining cold-start failure (0.038 vs. WP-small 0.185). With 20% target labels in the same joint training run (CP-transfer, Section 3.6), the scalar score calibrates to the target's ranking difficulty while still drawing on the full source signal, yielding a medium-effect profile ($d = 0.65$, $p < 0.001$ across all metrics) intermediate between COOBA (0.27) and BLAZE (1.16); this jointly-trained result is statistically equivalent to WP-large (TOST, ± 0.05 MRR), the joint signal closes the gap to the ample-data ceiling once target calibration is provided.

A particularly important cross-check is the models' chronological ordering: COOBA (2020) employs a more sophisticated adversarial mechanism than TRANP-CNN (2019), yet achieves worse outcomes across nearly all metrics. If model age or sophistication were the explanatory variable, COOBA should outperform TRANP-CNN; it does not, inconsistent with publication year and complexity as the primary explanation, and more consistent with evaluation mechanism as an explanatory dimension, though with only three models and their dimensions not independently manipulated, this remains an interpretation, not a controlled result (Section 3.2 defines "architecture" and which dimensions co-vary). Specifically, whether ranking at inference time requires target-side calibration of learned parameters appears to govern CPL viability: models ranking via pre-trained representation similarity (BLAZE) need no calibration and can operate in cold-start, while models ranking via a learned

scalar (TRANP-CNN, COOBA) require it and fail in cold-start, an inference-time mechanism independent of training-time sophistication.

5.3. Implications for CPL Research

The low cross-model agreement (44–62% across the three model pairs) is a methodological contribution for the CPL research community. It suggests that CPL benefit may not generalize across model architectures: a finding from one CPL model did not, in this corpus, reliably predict outcomes from a different architecture on the same project pairs. This echoes, in a different task, the central conclusion of the cross-project defect prediction literature introduced in Section 2: Hosseini et al. (2019)'s meta-analysis found transfer success in defect prediction to be context-dependent with no single reliably predictive factor, and our RQ2 null result for project-level metadata (Section 4) is external corroboration of the same pattern in bug localization.

The counter-intuitive negative predictor for source quality in COOBA, where better within-project performance predicts worse transfer, suggests that within-project performance and cross-project generalizability may be competing objectives for this model. High within-project performance may reflect overfitting to source-specific AST patterns that do not generalize; whether this extends to other GNN-based adversarial designs is untested here, since COOBA is our only instance of that paradigm. This motivates CPL-specific model design that explicitly prioritizes cross-project robustness over within-project peak performance, a direction we are not aware of existing proposals addressing. Notably, this tension does not appear in CNN-based or embedding-based rerankers, where source selection is essentially irrelevant to transfer outcome.

Beyond architectural choices, this study yields several observations about how future CPL evaluations should be designed, summarized in Section 6: reporting against a limited (not ample-data) within-project baseline, actually matching the target label budget between conditions (a control this study intended but did not achieve, as detailed above), and reporting negative-transfer rates and zero-label cold-start viability alongside win rates and mean gains. The cold-start result for BLAZE is among the most practically useful findings in this study, yet it is invisible under a standard fine-tuning-only evaluation protocol.

5.4. Threats to Validity

Internal validity. All three models depend on FAISS retrieval at the first stage. Recall at $K = 300$ varies across the 13 projects from 31.3% (ansible) to 100% (xarray), median $\approx 70\%$; three projects fall below 50% (ansible 31.3%, wagtail 34.6%, meson 45.5%), where the reranker never sees the ground-truth file in roughly half to two-thirds of queries. This imposes a shared performance ceiling that suppresses absolute metrics and conflates, in Table 8's large-codebase degradation, model limitation with retrieval ceiling. Disentangling the two would require re-ranking-only evaluation over gold-standard candidate sets, left to future

work. The within-model CPL comparison (CPT vs. WPS) is unaffected, since both conditions face the same constraint.

Training stochasticity and the fixed split. The fixed seed (42) controls the data splits, not the training process: weight initialization, data-loader ordering, and non-deterministic GPU kernels are unseeded, so identically configured WP-small runs on the same target differ by a mean min–max MRR spread of 0.05–0.07 across the three models (maximum 0.25), on the same order as the mean CPL gains, so *individual*-pair gains carry a substantial noise floor. Two properties protect the aggregate conclusions: the Wilcoxon analysis is paired at the run level, and re-testing CP-transfer against each target's *median* WP-small MRR preserves every RQ1 conclusion (BLAZE $p < 0.001$, $r_{rb} = 0.99$; TRANP-CNN $p < 0.001$, $r_{rb} = 0.74$; COOBA $p = 0.007$, $r_{rb} = 0.35$). Pair-level results, including the illustrative cases (Section 4.4), should be read with this noise floor in mind. The compute budget favored breadth of pairs over repeated seeds, since RQ3 needs many pairs evaluated identically across all three models; this trades per-pair variance estimation for architecture-level power, which the median re-test partially recovers.

Random rather than chronological splits. Splits are drawn at random (seed 42) rather than chronologically, which can leak future information: a test bug's training split may include later-filed bugs touching the same file, exposing a fix pattern that would not have existed at deployment time. We did not re-run the pipeline chronologically for this study; we flag it as an unaddressed threat and recommend a chronological sensitivity check before treating the random-split results as an upper bound on realistic deployment performance.

CP-transfer and WP-small are not matched on target-project labels. The orchestration script confirms WP-small draws 12.5% of the 80% target train-pool ($\approx 10\%$ of the full corpus) while CP-transfer draws 25% ($=20\%$), twice WP-small's share, e.g. 11 vs. 22 bugs for pydata/xarray and 133 vs. 267 for qiskit/qiskit, a consistent $\approx 2\times$ factor across all 13 targets. A CP-transfer advantage therefore cannot be attributed to cross-project knowledge alone: it may partly reflect the larger target-side set on top of the extra source data, and the two confounds are not distinguishable from each other or from a genuine transfer effect here. Disambiguating them needs either a WP-small re-run at CP-transfer's actual $\sim 20\%$ share or a shuffled-source condition, neither of which we ran; the CP-transfer-vs-WP-small gains reported throughout should accordingly be read as an upper bound on the cross-project-knowledge component, not an isolated measurement of it.

Construct validity. MRR and Top- K measure file-level localization; finer-grained (function- or line-level) transfer patterns are untested here, though the evaluation is aligned with prior CPL benchmarks.

Input-representation asymmetry. BLAZE reranks with modern frozen BGE embeddings (adapter-only fine-tuning); TRANP-CNN trains token embeddings from scratch and

COOBA uses GloVe-300, both faithful to their original designs. Representation quality differs across rerankers and is bundled into each architecture rather than controlled out. This is bounded by three observations: the candidate pool is identical for all models; the two non-BGE rerankers differ from each other ($d = 0.65$ vs. 0.27), so vintage alone does not determine CPL profile; and the cold-start dichotomy is consistent with the mechanism account in Section 5.2. A more specific asymmetry: the shared FAISS stage pools candidates via mean-pooled BGE embeddings, and BLAZE reranks that same pool with MaxSim over the same signal that selected it, while TRANSP-CNN and COOBA apply an independently-learned score to a pool chosen by a representation they never see. BLAZE thus re-scores its own retrieval signal with better aggregation rather than an independent signal, plausibly the strongest explanation for its cold-start advantage specifically; an untrained BGE+MaxSim baseline would isolate this aggregation effect from adapter training and is the most valuable follow-up addition.

Re-implementation fidelity. TRANSP-CNN and COOBA were re-implemented for this study (Section 3.2); loss functions, encoders, and adaptation mechanisms follow the originals, but the shared FAISS stage and, for TRANSP-CNN, the BGE tokenizer are departures, so absolute scores are not comparable to the original papers (within-study comparisons are unaffected). We cannot fully exclude COOBA's weak profile reflecting our re-implementation rather than the adversarial paradigm itself. We verified it against the original design equation-by-equation (bug encoder, feature extractors, gradient-reversal discriminator, margin-ranking loss) and found it faithful on every core mechanism, with three deliberate departures: BGE-code-v1 replaces GloVe, source and target task losses are summed in the same step rather than alternating per the original's Eq. 12, and evaluation uses one held-out split rather than 10-fold cross-validation. Re-running this re-implementation on the original paper's own benchmark data under a closer protocol is a scoped follow-up we did not carry out; until then the negative COOBA result should be attributed to the adversarial paradigm only provisionally.

Pre-training data leakage. BAAI/bge-code-v1 was pre-trained on a corpus that very likely includes these popular repositories (Aleithan et al., 2024), which may inflate BLAZE's cold-start result via memorized project-specific representations. A proxy check using GitHub star counts shows no significant Spearman correlation with cold-start MRR ($|\rho| < 0.20$), suggesting familiarity is not the primary driver, though a definitive test needs repositories absent from pre-training corpora. Separately, tangled commits (alongside the >10 -file patch filter applied during curation) introduce residual label noise as a construct threat.

Domain classification. The four domain labels used throughout RQ2 (Table 9) were assigned by the authors without a second rater or agreement statistic (Section 3.3 lists the most contestable calls: jupyterlab, ipython, prefect); a second pass with an inter-rater statistic (e.g. Cohen's κ)

should precede treating the domain-sensitivity results as final.

Empirical holdout validation is itself a data requirement. Recommending a small validation holdout (Section 5) is in tension with the zero-label framing motivating this study: a project able to construct one is not fully in that regime. We lack a principled minimum size; as a rough guide the smallest WP-small set here (11 bugs, pydata/xarray) still yielded a usable model, suggesting holdout validation becomes practical around a similar order of magnitude, roughly a dozen bugs, rather than at zero labels. Training and holdout sets serve different purposes, though, so this is only a rough analogy.

External validity. The study covers 13 Python projects; generalizability to other languages, proprietary codebases, or different bug-report styles is unvalidated. The Python scope is by design. The mechanism-based explanations (Section 5.2) are grounded in inference-time properties that are language-agnostic in principle, but COOBA's AST-based encoder is sensitive to identifier namespaces, call-graph depth, and type hierarchies that differ between Python and Java, so whether its failure mode replicates there is open. Only open-source projects with public commit histories are included, and the embedding backbone is specific to its evaluated version; a different backbone may shift absolute numbers while leaving relative conclusions intact. These constraints compound: the corpus favors popular, well-instrumented repositories that are also likely in the backbone's pre-training data, a combined selection effect to weigh when generalizing absolute figures (the within-model CPT-vs-WPS comparisons are less exposed).

No LLM-agent comparison. A reviewer may reasonably ask how these models compare against LLM-agent fault localization (Section 2); we cannot answer that here, since agents are evaluated on a different task and metric (patch generation over a shortlist) and no agent baseline exists at the Top- K /MRR/MAP metrics reported.

Conclusion validity. The cross-model consistency finding rests on pairs shared across all three models (Section 5.2 discusses TRANSP-CNN's intermediate effect size). Wilcoxon tests are conducted independently per metric with Bonferroni controlling the familywise error rate per five-metric family. The 63-pair set is structured rather than random, over-representing DS $\times$ DS pairs (Group A, 30 pairs); the RQ2 stratified analysis (Tables 8, 9) isolates within- from cross-domain effects. The 13 target projects recur unevenly (3–12 times each, median 3, mean 4.8; jupyterlab most often at 12, its Group B pivot role), so pairs are not independent observations. Beyond the median re-test above, we fit a linear mixed-effects model of CPL gain on model identity with target project as a random effect, following recent guidance for clustered observations (Gomes de Oliveira Neto et al., 2019). Model identity remains significant after accounting for the target-project effect (BLAZE intercept = 0.082; COOBA -0.071 , TRANSP-CNN -0.060 relative to BLAZE,

all $p < 0.001$), and the between-project variance ($= 0.001$) is smaller than, though not negligible beside, the residual ($= 0.003$; intraclass correlation ≈ 0.31), so clustering explains a real but minority share of the variation, with model identity dominant and the architecture effect robust to pair non-independence, though with only 13 clusters this estimate should be read cautiously.

6. Conclusion

Bug localization models require labeled training data that newly started or data-scarce projects often lack, and CPL has been proposed to close that gap, but to our knowledge no prior work has compared it against the practically relevant within-project-limited baseline or tested whether its findings generalize across model implementations. Every published CPL paper we are aware of evaluates one model against an ample-data comparator its intended users cannot reach. We addressed both gaps with a controlled study across 63 source→target pairs, 13 Python projects, and three architecturally distinct CPL models (embedding-based, convolutional, adversarial-GNN) under four scenarios isolating the contribution of cross-project knowledge.

Cross-project transfer outperforms within-project limited training for BLAZE and TRNP-CNN but yields only marginal benefit for COOBA, our only adversarial GNN instance and one we re-implemented, which also collapses to near-zero in cold-start (Section 4). BLAZE's zero-label deployment is statistically equivalent to WP-small and viable in nearly half of pairs; TRNP-CNN reaches full within-project performance with 20% target labels, the highest CPL ceiling relative to its own baseline, though BLAZE still leads on absolute MRR and Top-K throughout. CPL gain correlates strongly with how few labels the target already has, the least-labeled targets, the ones this study is written for, benefit the most. This benefit has limits, however: negative transfer affects 31.7% of COOBA deployments, 15.9% of TRNP-CNN's, and 1.6% of BLAZE's; domain sensitivity is steepest for TRNP-CNN and absent for COOBA; similarity-based project features do not reliably predict pair-level success, though target label count does; and cross-model direction agreement ranges from only 44% to 62%, so a result from one architecture does not reliably predict another's even on the same pair, though BLAZE's and TRNP-CNN's gains are themselves correlated (Section 5).

For practitioners, BLAZE can often be deployed immediately with zero target annotations, climbing to a 90.0% within-domain win rate once a modest label set is added; TRNP-CNN needs 20% of target labels to match full within-project training. Both carry a cost WP-small does not: CP-transfer trains on the full source corpus plus twice WP-small's target-side data ($\sim 20\%$ vs. $\sim 10\%$, Section 5.4), and neither guarantees a win. Source and domain alignment should be checked empirically, not assumed, particularly for TRNP-CNN and COOBA.

Future work should pursue: two evaluation-only additions that isolate confounds identified in Section 5.4, an

untrained BGE-with-MaxSim baseline to separate the aggregation effect from adapter training, and BM25 as a zero-label comparator to measure cold-start viability against the full set of options open to an unlabelled project rather than against WP-small alone (Section 3.5), neither of which requires model training; extending to multi-language CPL (Java, cross-language pairs, Kotlin/Go) to test whether these findings generalize beyond Python (Yang et al., 2024a); model-specific source-selection criteria beyond project metadata; and function- or line-level localization.

The most durable contribution of this study, beyond any single number, is methodological: low cross-model agreement suggests CPL benefit may not generalize across architectures, so future CPL evaluations should report results per-architecture, against a limited (not ample-data) baseline, with the target label budget *verified as* matched between conditions, our own CP-transfer and WP-small scenarios are not matched on target-project labels (Section 5.4), a discrepancy worth checking for in any similarly-structured design, and with negative-transfer rates reported alongside win rates and mean gains.

CRedit authorship contribution statement

A Eashaan Rao: Conceptualization, Methodology, Software, Data curation, Investigation, Formal analysis, Visualization, Writing – original draft. **Sridhar Chimalakonda:** Conceptualization, Methodology, Supervision, Writing – review & editing.

Declaration of competing interest

The authors declare that they have no known competing financial interests or personal relationships that could have appeared to influence the work reported in this paper.

Funding

This work was supported by the Indian Institute of Technology Tirupati. The funder had no role in study design, data collection and analysis, decision to publish, or preparation of the manuscript.

Declaration of generative AI and AI-assisted technologies in the writing process

During the preparation of this work, the authors used AI-assisted tools (RISHA Lab, IIT Tirupati) to improve the clarity, presentation, and language of the writing. After using this tool, the authors reviewed and edited the content as needed and take full responsibility for the content of the publication. AI tools were not used to generate the study's research ideas, experimental design, analysis, or conclusions.

Acknowledgements

This work was supported by the Indian Institute of Technology Tirupati.

Data Availability

The replication package for this study is publicly archived on Zenodo at <https://doi.org/10.5281/zenodo.21560518>. It includes the curated bug metadata databases for all projects, experiment result CSVs, the four-scenario evaluation pipeline, and all analysis scripts. The package supports full reproduction of Tables 6–11 and all figures in Section 4. Given the re-implementation fidelity threat discussed in Section 5.4, the package explicitly includes our COOBA and TRANP-CNN re-implementations (source, configuration, and the equation-by-equation verification notes against the original publications), alongside the BLAZE code used as published, so that the re-implemented components can be independently audited. The ACM SIGSOFT empirical standards checklist accompanying this study (Section 3) is also included in the package.

References

- Aleithan, R., Xue, H., Mohajer, M.M., Nnorom, E., Uddin, G., Wang, S., 2024. SWE-Bench+: Enhanced coding benchmark for LLMs. *arXiv:2410.06992*.
- Arcuri, A., Briand, L., 2011. A practical guide for using statistical tests to assess randomized algorithms in software engineering, in: *Proceedings of the 33rd International Conference on Software Engineering (ICSE)*, ACM. pp. 1–10.
- Arcuri, A., Briand, L., 2014. A hitchhiker's guide to statistical tests for assessing randomized algorithms in software engineering. *Software Testing, Verification and Reliability* 24, 219–250.
- Beijing Academy of Artificial Intelligence, 2025. BAAI/bge-code-v1: A code-specialized text embedding model. <https://huggingface.co/BAAI/bge-code-v1>. Accessed: 2025.
- Ben-David, S., Blitzer, J., Crammer, K., Kulesza, A., Pereira, F., Vaughan, J.W., 2010. A theory of learning from different domains. *Machine Learning* 79, 151–175.
- Bogomolov, E., Eliseeva, A., Galimzyanov, T., Glukhov, T., Shapkin, I., Tiganov, M., Golubev, Y., Ivanov, V., Sokolov, N., Evtikhiev, M., et al., 2024. Long code arena: a set of benchmarks for long-context code models, in: *Proceedings of the 2024 Conference on Empirical Methods in Natural Language Processing*.
- Chakraborty, P., Alfadil, M., Nagappan, M., 2025. BLAZE: Cross-language and cross-project bug localization via dynamic chunking and hard example learning. *IEEE Transactions on Software Engineering*.
- Chandramohan, M., Nguyen, D.Q., Krishnan, P., Jancic, J., 2024. Supporting cross-language cross-project bug localization using pre-trained language models. *arXiv:2407.02732*.
- Ciborowska, A., Damevski, K., 2022. Fast changeset-based bug localization with BERT, in: *Proceedings of the 44th International Conference on Software Engineering*, ACM. pp. 2666–2677.
- Cohen, J., 1988. *Statistical Power Analysis for the Behavioral Sciences*. 2nd ed., Lawrence Erlbaum Associates, Hillsdale, NJ.
- Devlin, J., Chang, M.W., Lee, K., Toutanova, K., 2019. BERT: Pre-training of deep bidirectional transformers for language understanding, in: *Proceedings of the 2019 Conference of the North American Chapter of the Association for Computational Linguistics (NAACL-HLT)*, pp. 4171–4186.
- Feng, Z., Guo, D., Tang, D., Duan, N., Feng, X., Gong, M., Shou, L., Qin, B., Liu, T., Jiang, D., Zhou, M., 2020. CodeBERT: A pre-trained model for programming and natural languages, in: *Findings of the Association for Computational Linguistics: EMNLP 2020*, pp. 1536–1547.
- Ganin, Y., Ustinova, E., Ajakan, H., Germain, P., Larochelle, H., Laviolette, F., Marchand, M., Lempitsky, V., 2016. Domain-adversarial training of neural networks. *Journal of Machine Learning Research* 17, 1–35.
- Giray, G., Bennin, K.E., Köksal, Ö., Babur, Ö., Tekinerdogan, B., 2023. On the use of deep learning in software defect prediction. *Journal of Systems and Software* 195, 111537.
- Gomes de Oliveira Neto, F., Torkar, R., Feldt, R., Gren, L., Furia, C.A., Huang, Z., 2019. Evolution of statistical analysis in empirical software engineering research: Current state and steps forward. *Journal of Systems and Software* 156, 246–267.
- Guo, D., Lu, S., Duan, N., Wang, Y., Zhou, M., Yin, J., 2022. UniXcoder: Unified cross-modal pre-training for code representation, in: *Proceedings of the 60th Annual Meeting of the Association for Computational Linguistics (ACL)*, pp. 7212–7225.
- Guo, D., Ren, S., Lu, S., Feng, Z., Tang, D., Liu, S., Zhou, L., Duan, N., Svyatkovskiy, A., Fu, S., Tufano, M., Deng, S.K., Clement, C., Drain, D., Sundaresan, N., Yin, J., Jiang, D., Zhou, M., 2021. GraphCodeBERT: Pre-training code representations with data flow, in: *International Conference on Learning Representations (ICLR)*.
- Hirsch, T., Hofer, B., 2022. A systematic literature review on benchmarks for evaluating debugging approaches. *Journal of Systems and Software* 192, 111423.
- Hossain, S.B., Jiang, N., Zhou, Q., Li, X., Chiang, W.H., Lyu, Y., Nguyen, H., Tripp, O., 2024. A deep dive into large language models for automated bug localization and repair. *Proceedings of the ACM on Software Engineering* 1.
- Hosseini, S., Turhan, B., Gunarathna, D., 2019. A systematic literature review and meta-analysis on cross project defect prediction. *IEEE Transactions on Software Engineering* 45, 111–147.
- Huo, X., Thung, F., Li, M., Lo, D., Shi, S.T., 2021. Deep transfer bug localization. *IEEE Transactions on Software Engineering* 47, 1368–1380.
- Jimenez, C.E., Yang, J., Wetteg, A., Yao, S., Pei, K., Press, O., Narasimhan, K., 2024. SWE-bench: Can language models resolve real-world GitHub issues?, in: *The Twelfth International Conference on Learning Representations*.
- Johnson, J., Douze, M., Jégou, H., 2021. Billion-scale similarity search with GPUs. *IEEE Transactions on Big Data* 7, 535–547.
- Kalliamvakou, E., Gousios, G., Blincoc, K., Singer, L., German, D.M., Damian, D., 2014. The promises and perils of mining GitHub, in: *Proceedings of the 11th Working Conference on Mining Software Repositories (MSR)*, ACM. pp. 92–101.
- Kampenes, V.B., Dybå, T., Hannay, J.E., Sjøberg, D.I., 2007. A systematic review of effect size in software engineering experiments. *Information and Software Technology* 49, 1073–1086.
- Kendall, M.G., 1938. A new measure of rank correlation. *Biometrika* 30, 81–93.
- Kim, K., Ghatpande, S., Liu, K., Koyuncu, A., Kim, D., Bissyandé, T.F., Klein, J., Le Traon, Y., 2022. DigBug—pre/post-processing operator selection for accurate bug localization. *Journal of Systems and Software* 189, 111300.
- Lakens, D., 2017. Equivalence tests: A practical primer for t tests, correlations, and meta-analyses. *Social Psychological and Personality Science* 8, 355–362.
- Lam, A.N., Nguyen, A.T., Nguyen, H.A., Nguyen, T.N., 2017. Bug localization with combination of deep learning and information retrieval, in: *2017 IEEE/ACM 25th International Conference on Program Comprehension (ICPC)*, IEEE. pp. 218–229.
- Lee, J., Kim, D., Bissyandé, T.F., Jung, W., Le Traon, Y., 2018. Bench4BL: Reproducibility study on the performance of IR-based bug localization, in: *Proceedings of the 27th ACM SIGSOFT International Symposium on Software Testing and Analysis*, pp. 61–72.
- Li, Y., Liu, B., Zhang, T., Wang, Z., Lo, D., Yang, L., Lyu, J., Zhang, H., 2025. A knowledge enhanced large language model for bug localization. *Proceedings of the ACM on Software Engineering* 2, 1914–1936.
- Liang, H., Hang, D., Li, X., 2022. Modeling function-level interactions for file-level bug localization. *Empirical Software Engineering* 27, 186.
- Niu, F., Li, C., Liu, K., Xia, X., Lo, D., 2025. When deep learning meets information retrieval-based bug localization: A survey. *ACM Computing Surveys* 57, 1–41.

- van den Oord, A., Li, Y., Vinyals, O., 2018. Representation learning with contrastive predictive coding. *arXiv:1807.03748*.
- Pan, S.J., Yang, Q., 2010. A survey on transfer learning. *IEEE Transactions on Knowledge and Data Engineering* 22, 1345–1359.
- Ralph, P., et al., 2021. Empirical standards for software engineering research. *arXiv:2010.03525*. aCM SIGSOFT Empirical Standards initiative.
- Reddy, R.G., Suresh, T., Doo, J., Liu, Y., Nguyen, X.P., Zhou, Y., Yavuz, S., Xiong, C., Ji, H., Joty, S., 2025. SweRank: Software issue localization with code ranking. *arXiv:2505.07849*.
- Reimers, N., Gurevych, I., 2019. Sentence-BERT: Sentence embeddings using Siamese BERT-networks, in: *Proceedings of the 2019 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 3982–3992.
- Robertson, S., Zaragoza, H., 2009. The probabilistic relevance framework: BM25 and beyond. *Foundations and Trends in Information Retrieval* 3, 333–389.
- Saha, R.K., Lease, M., Khurshid, S., Perry, D.E., 2013. Improving bug localization using structured information retrieval, in: *2013 28th IEEE/ACM International Conference on Automated Software Engineering (ASE)*, IEEE. pp. 345–355.
- Schuurmann, D.J., 1987. A comparison of the two one-sided tests procedure and the power approach for assessing the equivalence of average bioavailability. *Journal of Pharmacokinetics and Biopharmaceutics* 15, 657–680.
- Sotto-Mayor, B., Kalech, M., 2024. A survey on transfer learning for cross-project defect prediction. *IEEE Access* 12, 93398–93425.
- Spearman, C., 1904. The proof and measurement of association between two things. *American Journal of Psychology* 15, 72–101.
- Tong, H., Lu, W., Xing, W., Wang, S., 2023. ARRAY: Adaptive triple feature-weighted transfer Naive Bayes for cross-project defect prediction. *Journal of Systems and Software* 202, 111721.
- Wang, C., Li, Y., Chen, L., Huang, W., Zhou, Y., Xu, B., 2020. Examining the effects of developer familiarity on bug fixing. *Journal of Systems and Software* 169, 110667.
- Wang, Y., Wang, W., Joty, S., Hoi, S.C., 2021. CodeT5: Identifier-aware unified pre-trained encoder-decoder models for code understanding and generation, in: *Proceedings of the 2021 Conference on Empirical Methods in Natural Language Processing (EMNLP)*, pp. 8696–8708.
- Wilcoxon, F., 1945. Individual comparisons by ranking methods. *Biometrics Bulletin* 1, 80–83.
- Xia, C.S., Deng, Y., Dunn, S., Zhang, L., 2024. Agentless: Demystifying LLM-based software engineering agents. *arXiv preprint arXiv:2407.01489*.
- Yang, H., Nong, Y., Wang, S., Cai, H., 2024a. Multi-language software development: Issues, challenges, and solutions. *IEEE Transactions on Software Engineering* 50, 512–533.
- Yang, J., Jimenez, C.E., Wettig, A., Lieret, K., Yao, S., Narasimhan, K., Press, O., 2024b. SWE-agent: Agent-computer interfaces enable automated software engineering, in: *Advances in Neural Information Processing Systems*.
- Ye, X., Bunescu, R., Liu, C., 2014. Learning to rank relevant files for bug reports using domain knowledge, in: *Proceedings of the 22nd ACM SIGSOFT International Symposium on Foundations of Software Engineering*, pp. 689–699.
- Yu, Q., Jiang, S., Zhang, Y., 2017. A feature matching and transfer approach for cross-company defect prediction. *Journal of Systems and Software* 132, 366–378.
- Zhang, W., Deng, L., Zhang, L., Wu, D., 2022. A survey on negative transfer. *IEEE/CAA Journal of Automatica Sinica* 10, 305–329.
- Zhang, Y., Ruan, H., Fan, Z., Roychoudhury, A., 2024. AutoCodeRover: Autonomous program improvement, in: *Proceedings of the 33rd ACM SIGSOFT International Symposium on Software Testing and Analysis (ISSTA)*, pp. 1592–1604.
- Zhou, J., Zhang, H., Lo, D., 2012. Where should the bugs be fixed? More accurate information retrieval-based bug localization based on bug reports, in: *2012 34th International Conference on Software Engineering (ICSE)*, IEEE. pp. 14–24.
- Zhu, Z., Li, Y., Tong, H., Wang, Y., 2020. COOBA: Cross-project bug localization via adversarial transfer learning, in: *Proceedings of the 29th International Joint Conference on Artificial Intelligence (IJCAI)*.
- Zhu, Z., Peng, L., Wang, Y., Li, Y., Xianzhong, L., 2025. FunLoc: A novel function-level bug localization framework enhanced by contrastive and active learning strategies, in: *Proceedings of the 34th ACM International Conference on Information and Knowledge Management (CIKM)*.
- Zhu, Z., Tong, H., Wang, Y., Li, Y., 2023. BL-GAN: Semi-supervised bug localization via generative adversarial network. *IEEE Transactions on Knowledge and Data Engineering* 35, 11112–11125.
- Zhu, Z., Wang, Y., Li, Y., 2021. TroBo: A novel deep transfer model for enhancing cross-project bug localization, in: *Knowledge Science, Engineering and Management (KSEM)*, Springer. pp. 556–568.